

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI Agent 개요



CONTENTS

목차

- 01 에이전트 동작 원리: 도구 호출 루프
- 02 랭체인 기초: 부품과 체인
- 03 RAG: 검색기 구축
- 04 RAG 에이전트 서비스
- 05 RAG와 LLM wiki

과정 전체 지도



이 강의에서 쓰는 모델 호출과 파이썬 최소 지식

모델 호출 (litellm completion)

모델명과 메시지 목록을 주면 응답 객체가 돌아옴.

```
completion(model=MODEL,
messages=msgsg)
```

메시지 (message)

역할(role)과 내용(content)을 담은 dict. 대화 기록은 이런 dict를 순서대로 담은 리스트.

```
{"role": "user", "content": "질문"}
```

조기 반환 (early return)

조건이 맞으면 남은 줄을 건너뛰고 함수를 그 자리에서 끝냄.

```
if not msg.tool_calls: return
```

pydantic 스키마 (BaseModel)

필드 이름과 타입을 선언한 클래스. 어긋난 값은 검증 에러가 남.

```
class Answer(BaseModel): topic: str
```

JSON 파싱 (json.loads)

JSON 문자열을 파이썬 dict로 바꿈. 형식이 어긋나면 그 자리에서 에러.

```
json.loads('{"city": "서울"}')
```

키워드 인자 언패킹 (**)

dict를 함수의 이름 붙은 인자로 펼쳐 넣음.

```
fn(**{"topic": "환불"})
```

API 키 관리: 코드 밖으로

- API 키는 **비밀번호와 같은 자격증명**.
코드·저장소에 들어가는 순간 유출 경로가 생김.
- 표준 관행: 키는 **.env 파일**에만 두고,
코드는 환경변수로 읽음. .env는 저장소에 올리지
않음
(.gitignore 등록).
- 개인 GitHub에 commit 하기 전 점검:
키 문자열이 코드·커밋에 없는가.

키가 파일로 굴러다니는 꼴

api_key.txt를 소스 옆에 두고 읽음

파일째 복사·압축·공유되며 퍼짐. 저장소에
커밋되면 이력에 영구히 남음.

환경변수로 공급하는 꼴

.env + os.environ (.gitignore 등록)

코드와 자격증명이 분리됨. 저장소에는 키 자리만
비운 .env 견본을 둬.

Agent 개요

에이전트 동작 원리: 도구 호출 루프

도구 호출 루프 / 도구 스키마 / 손 루프(agent loop, from scratch) / 추론 패턴

학습 목표

- 도구 호출(tool use) 루프의 네 단계(대화 기록 초기화 → 도구 목록과 함께 모델 호출 → 도구 호출 여부 판정 → 실행 결과 되먹임)를, 루프 코드의 해당 줄을 짚어 가며 설명할 수 있습니다.
- 함수 1개에 이름, 설명, 파라미터 스키마를 붙여 모델에 건네고, 프레임워크 없이 루프를 완주시키는 코드를 작성할 수 있습니다.
- 도구 실행이 실패했을 때 에러 메시지를 도구 결과로 되먹여 모델이 스스로 재시도하게 만드는 처리를, 기존 코드에 추가할 수 있습니다.
- ReAct·Plan-and-Execute·ToT의 계획 시점 차이를 설명할 수 있습니다.

▶ 에이전트 동작 원리

랭체인 기초

RAG: 검색기 구축

RAG 에이전트 서비스

RAG와 LLM wiki

LLM 에이전트의 구성



시각을 묻는 도구 없는 호출

【질문】 지금 몇 시인가요? 정확한 현재 시각을 알려 주세요.

【응답】 정확한 현재 시각은 제가 실시간 시계를 확인할 수 없어 알려드리기 어렵습니다. 현재 날짜는 **2026년 8월 25일**로 표시되어 있습니다. 휴대전화나 컴퓨터의 시계를 확인해 주세요.

- 모델이 만드는 것은 텍스트뿐.
시계를 읽는 실행 수단은 모델 안에 없음.
- 답을 지어내지 않고 확인할 수 없다고 응답함.
없는 것은 능력이 아니라 손.
- 모델에 손을 달아 주면 어떻게 되는가.

문제상황과 목표

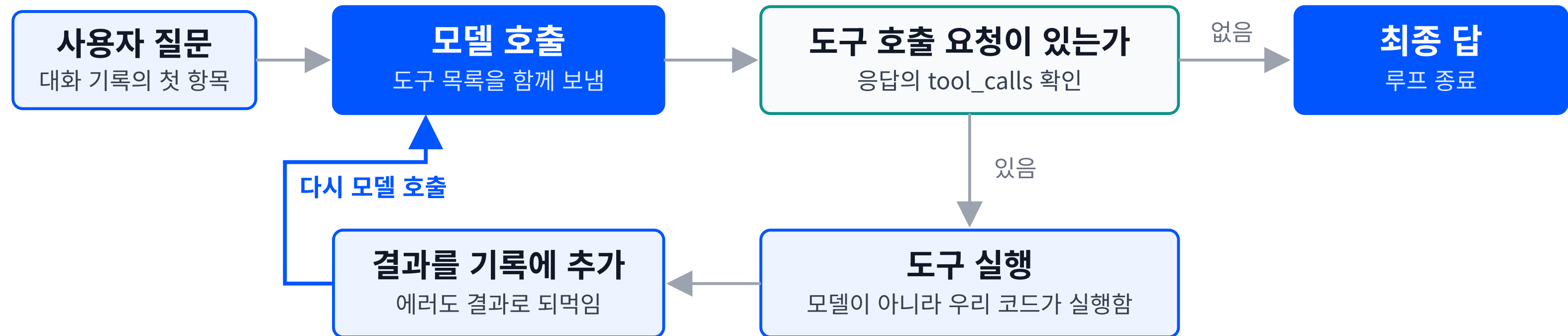
문제

구름월드 놀이공원 고객센터 담당자예요. "지금 몇 시예요, 주차비는요, 환불은 어떻게 해요" 같은 질문이 하루 종일 들어와요.

목표

고객센터에 오는 질문을 받으면 FAQ와 현재 시각을 스스로 찾아본 뒤 답하는 안내 프로그램을 만들어봅시다.

문제해결을 위한 워크플로우 설계(예시)



반복 한 바퀴가 모델 호출 한 번 · 종료는 「도구 호출 요청 없음」 한 곳뿐

각 단계별 요구사항

단계	무엇을 받아	어떻게	무엇을 넘긴다
사용자 질문	질문 한 줄을	대화 기록의 첫 항목으로 삼아	모델에 넘긴다
모델 호출	고정 지침(시스템 프롬프트) 없이 대화 이력과 도구 목록만	답할지 도구를 부를지 스스로 정해	응답을 넘긴다
도구 호출 요청이 있는가	응답을	도구 요청이 들어 있는지 보고	있으면 도구 실행으로, 없으면 최종 답으로 보낸다
도구 실행	요청된 도구와 인자를	우리 코드가 그 도구를 실행해	결과를 넘긴다
결과를 기록에 추가	도구 결과를	오류도 결과로 삼아 대화 기록에 붙이고	다시 모델 호출로 보낸다
최종 답	도구 요청이 없는 응답을	그것을 답으로 삼아	루프를 끝낸다

도구 호출 루프(tool use loop): 모델 호출 →
도구 호출 요청 판정 → 도구 실행 →
결과를 대화 기록에 추가 → 재호출.

종료 조건은 둘. 도구 호출이 없는 응답(최종 답), 또는 반복 한도 도달.

도구 스키마(function calling): 이름, 설명, 파라미터

```
{
  "type": "function",
  "function": {
    "name": "faq_lookup",
    "description":
      "시설 FAQ에서 항목을 조회한다. "
      "운영시간, 주차, 환불 질문에 쓴다.",
    "parameters": {
      "type": "object",
      "properties": {
        "topic": {
          "type": "string",
          "description":
            "조회할 항목 이름."
        },
        "required": ["topic"]}
    }
  }
}
```

- **이름(name)**: 모델이 호출 요청에 적어 보내는 식별자. 우리 코드가 이 이름으로 함수를 찾음.
- **설명(description)**: 무엇을 하는 도구이고 언제 쓰는지를 적는 자연어. 모델이 도구를 고를 때 읽는 유일한 근거.
- **파라미터(parameters)**: 인자의 이름, 타입, 필수 여부를 JSON Schema로 명세함.

설명의 질이 도구 선택의 질을 결정함.

설명 문안의 정오 대비

판단 근거가 없는 설명

```
"name": "faq_lookup",  
"description": "조회한다."
```

- 무엇을 조회하는지, 어떤 질문에 쓰는지가 문안에 없음.
- 도구가 여럿이면 **고를 근거가 사라짐.**

판단 근거가 있는 설명

```
"name": "faq_lookup",  
"description": "시설 FAQ에서 항목을  
조회한다. 운영시간, 주차, 환불  
질문에 쓴다."
```

- 대상(시설 FAQ), 동작(조회), 적용 범위(운영시간, 주차, 환불)가 문안에 들어 있음.
- 코드를 고치지 않고 **문안만으로** 선택 근거를 바꿀 수 있음.

도구 두 개 등록

```
FAQ = {
    "운영시간": "매일 09:30~21:00에 운영합니다.",
    "주차": "주차장은 4,000대 규모이며 "
            "최초 30분은 무료입니다.",
    "환불": "이용일 전날까지 전액 환불, "
            "당일은 50% 환불입니다.",
}

def faq_lookup(topic: str) -> str:
    return FAQ[topic]
def get_now() -> str:
    return datetime.now().strftime(
        "%Y-%m-%d %H:%M")
TOOLS = {"faq_lookup": faq_lookup,
         "get_now": get_now}
```

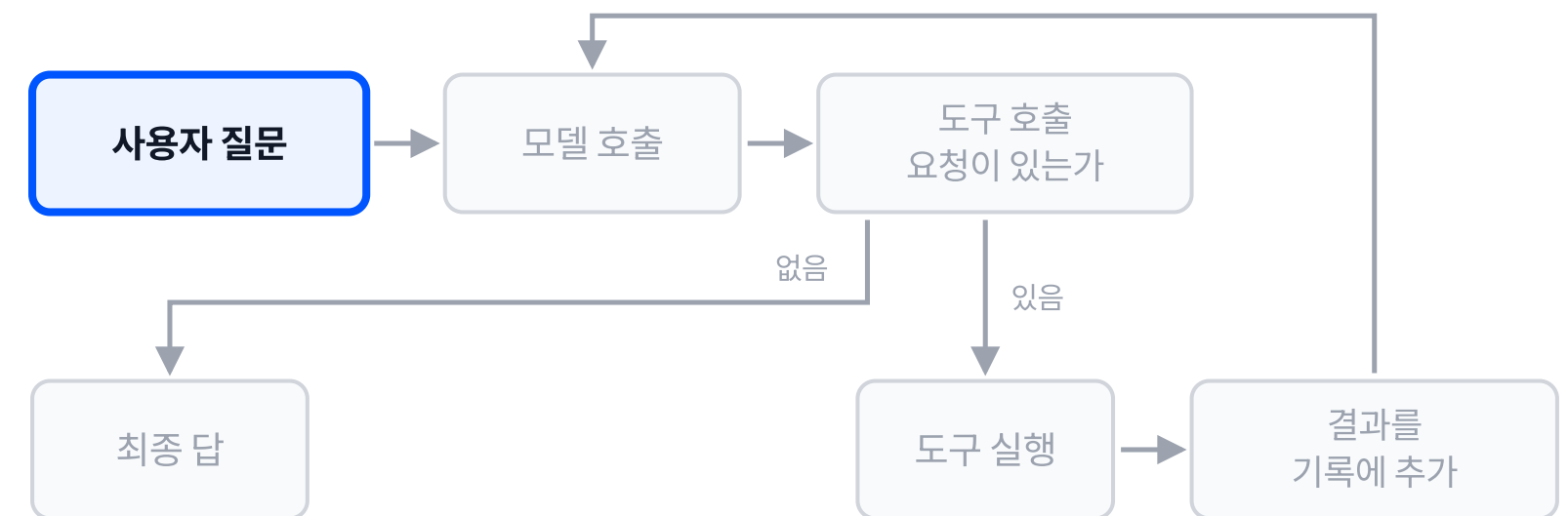
구름월드 FAQ 안내의 도구 두 개

도구 이름	하는 일	파라미터
faq_lookup	FAQ 항목 조회	topic
get_now	현재 날짜와 시각	없음

- 도구의 실체는 평범한 파이썬 함수.
스키마는 그 함수를 모델에게 소개하는 명세.
- **TOOLS**는 도구 이름과 실제 함수를 짝지은 표.
모델이 보낸 이름을 여기서 함수로 바꿈.

1단계: 대화 기록 초기화

```
def run_agent(question, max_turn=4):
    messages = [user_msg(question)]
    for _ in range(max_turn):
        res = completion(
            model=MODEL,
            messages=messages,
            tools=TOOL_SCHEMA)
        msg = res.choices[0].message
        if not msg.tool_calls:
            return msg.content
        messages.append(msg)
        for call in msg.tool_calls:
            fn = TOOLS[call.function.name]
            args = json.loads(
                call.function.arguments)
            messages.append(tool_msg(
                call.id, fn(**args)))
    return "반복 한도 초과"
```



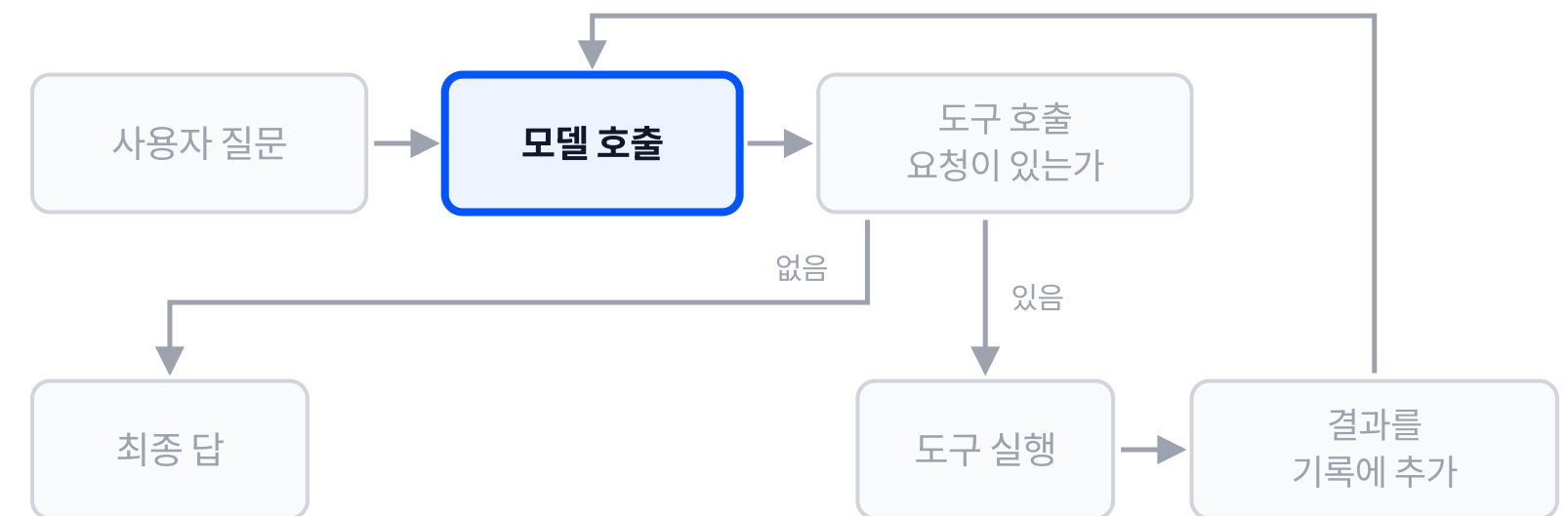
messages 리스트 하나가 대화 전체의 기록. 질문이 그 첫 항목으로 들어감.

max_turn은 반복 한도. 도구 호출이 끝나지 않아도 정해진 횟수에서 멈춤.

이 코드는 프레임워크 없이 litellm **completion**을 직접 부름.

2단계: 도구 목록과 함께 모델 호출

```
def run_agent(question, max_turn=4):
    messages = [user_msg(question)]
    for _ in range(max_turn):
        res = completion(
            model=MODEL,
            messages=messages,
            tools=TOOL_SCHEMA)
        msg = res.choices[0].message
        if not msg.tool_calls:
            return msg.content
        messages.append(msg)
        for call in msg.tool_calls:
            fn = TOOLS[call.function.name]
            args = json.loads(
                call.function.arguments)
            messages.append(tool_msg(
                call.id, fn(**args)))
    return "반복 한도 초과"
```



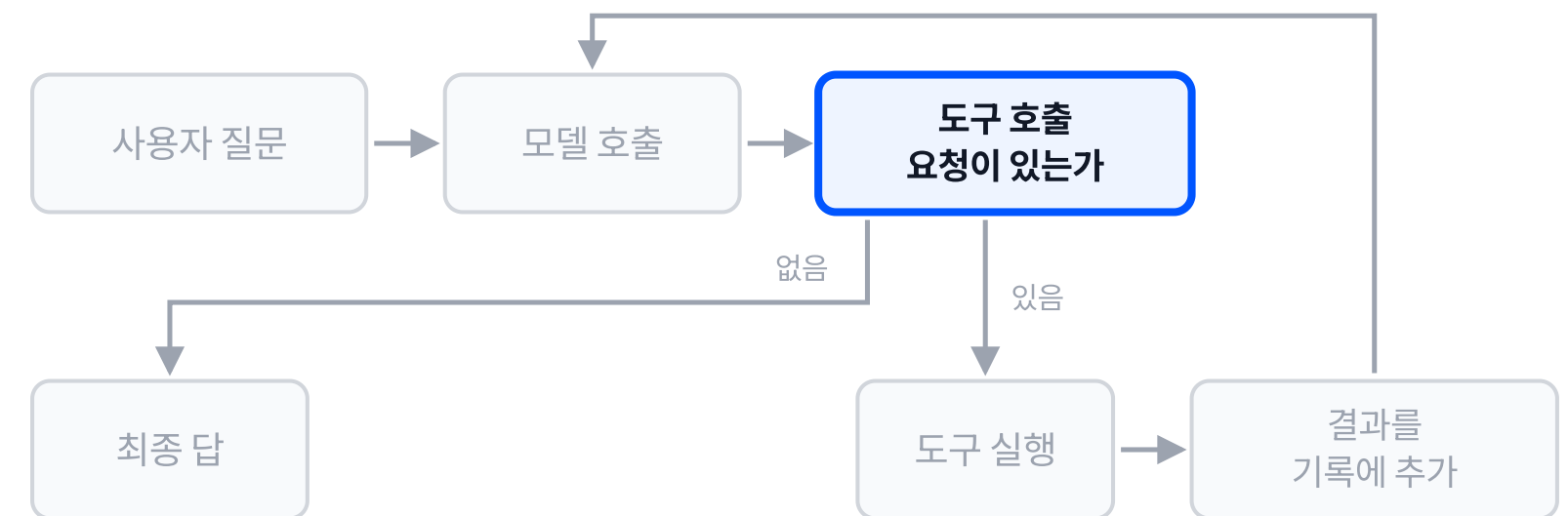
도구 목록 **TOOL_SCHEMA**를 호출마다 함께 보냄. 모델은 이 목록 안에서만 도구를 고름.

messages는 누적본이라 직전까지의 도구 결과가 전부 실려 나감.

반복문 한 바퀴가 모델 호출 한 번.

3단계: 도구 호출 여부 판정

```
def run_agent(question, max_turn=4):
    messages = [user_msg(question)]
    for _ in range(max_turn):
        res = completion(
            model=MODEL,
            messages=messages,
            tools=TOOL_SCHEMA)
        msg = res.choices[0].message
        if not msg.tool_calls:
            return msg.content
        messages.append(msg)
        for call in msg.tool_calls:
            fn = TOOLS[call.function.name]
            args = json.loads(
                call.function.arguments)
            messages.append(tool_msg(
                call.id, fn(**args)))
    return "반복 한도 초과"
```



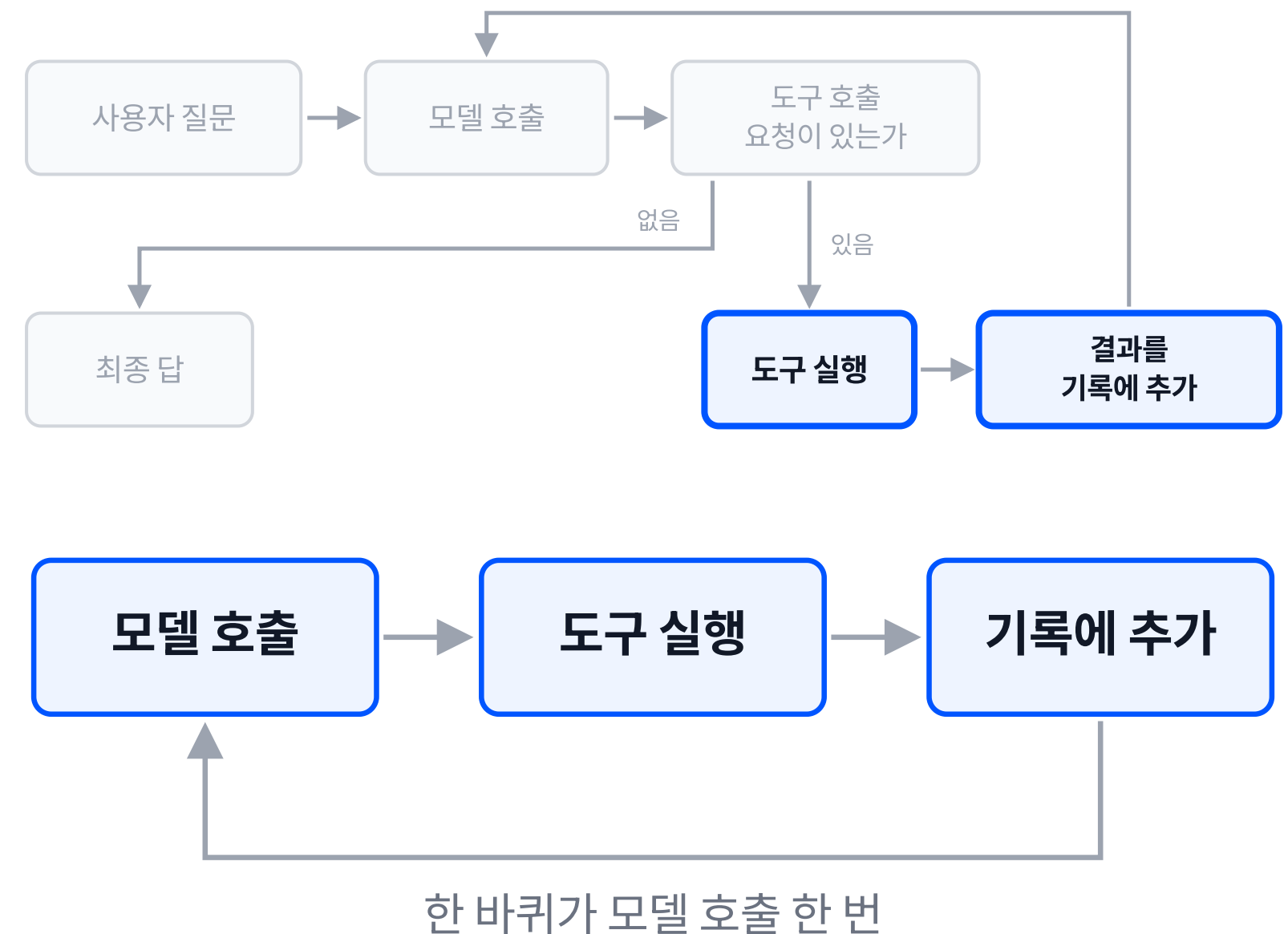
tool_calls가 비어 있으면 모델이 최종 답을 만든 상태. 그 자리에서 반환함.

비어 있지 않으면 모델의 요청을 기록에 먼저 붙임. 요청과 결과가 짝을 이루어야 함.

반복을 끝내는 자리는 이 한 곳뿐.

4단계: 도구 실행 결과 되먹임

```
def run_agent(question, max_turn=4):
    messages = [user_msg(question)]
    for _ in range(max_turn):
        res = completion(
            model=MODEL,
            messages=messages,
            tools=TOOL_SCHEMA)
        msg = res.choices[0].message
        if not msg.tool_calls:
            return msg.content
        messages.append(msg)
        for call in msg.tool_calls:
            fn = TOOLS[call.function.name]
            args = json.loads(
                call.function.arguments)
            messages.append(tool_msg(
                call.id, fn(**args)))
    return "반복 한도 초과"
```



실행 트레이스(trace): 질문에서 답까지

[질문] 운영시간이 어떻게 되나요?

[도구 호출] `faq_lookup({'topic': '운영시간'})`

[도구 결과] 매일 09:30~21:00에 운영합니다.

[최종 답] 운영시간은 **매일 09:30~21:00**입니다.

출력 줄	루프의 단계
[질문]	1. 대화 기록 초기화
[도구 호출]	2~3. 모델 호출과 판정
[도구 결과]	4. 도구 실행과 되먹임
[최종 답]	3. 도구 호출 없음, 반환

- 모델이 보낸 것은 실행 결과가 아니라 **호출 요청**. 실행은 우리 코드가 함.
- 반복문이 두 바퀴 돌았고, 두 번째 바퀴에서 도구 호출이 없어 최종 답으로 끝남.

질문에 따라 달라지는 도구 선택

FAQ 질문

[질문] 운영시간이 어떻게 되나요?

[도구 호출] `faq_lookup({'topic': '운영시간'})`

[도구 결과] 매일 09:30~21:00에 운영합니다.

[최종 답] 운영시간은 **매일 09:30~21:00**입니다.

시각 질문

[질문] 지금 몇 시인가요?

[도구 호출] `get_now({})`

[도구 결과] 2026-08-25 20:34

[최종 답] 현재 **2026년 8월 25일 오후 8시 34분**입니다.

- 코드는 두 경우가 완전히 같음. 바뀐 것은 질문뿐.
- 선택의 근거는 도구 목록에 실린 **이름과 설명**이고, 인자도 모델이 채워 보냄.

대화 기록에 쌓이는 네 메시지

```
def user_msg(q):  
    return {"role": "user", "content": q}  
  
def tool_msg(call_id, content):  
    return {"role": "tool",  
            "tool_call_id": call_id,  
            "content": str(content)}
```

- 메시지마다 **role**이 붙음. 모델은 이 역할을 보고 누가 한 말인지 구분함.
- 도구 결과에는 **tool_call_id**가 붙어, 어느 요청에 대한 답인지 짝이 유지됨.

순서	role	기록되는 내용
1	user	운영시간이 어떻게 되나요?
2	assistant	faq_lookup 호출 요청
3	tool	매일 09:30~21:00에 운영합니다.
4	assistant	최종 답

대화 기록이 곧 에이전트의 상태.
루프가 도는 동안 이 리스트만 자람.

한 응답에 실린 도구 호출 두 건

[질문] 지금 시각이랑 환불 규정을 같이 알려 주세요.

[도구 호출] `get_now({})`

[도구 결과] 2026-08-25 20:34

[도구 호출] `faq_lookup({'topic': '환불'})`

[도구 결과] 이용일 전날까지 전액 환불,
당일은 50% 환불입니다.

[최종 답] 현재 시각은 **2026년 8월 25일
20시 34분**입니다.

환불 규정은 다음과 같습니다.

- **이용일 전날까지:** 전액 환불
- **이용 당일:** 50% 환불

- **tool_calls**는 리스트. 한 응답에 호출이 여러 건 실릴 수 있음(병렬 도구 호출, parallel tool calling).
- 두 건 모두 실행되고, 두 결과가 모두 기록에 쌓인 뒤 다음 모델 호출이 일어남.
- 도구 호출이 두 건이어도 **모델 호출은 두 번**(요청·최종 답)으로, 도구가 한 건일 때와 같음.

한 번의 응답이 여러 건의 작업 요청을 담음.

여러 건의 호출을 처리하는 실행부

```
for call in msg.tool_calls:
    fn = TOOLS[call.function.name]
    args = json.loads(
        call.function.arguments)
    result = fn(**args)
    messages.append(
        tool_msg(call.id, result))
```

- 실행부는 **리스트를 위에서부터** 차례로 훑는 반복문. 호출이 몇 건이든 같은 코드가 처리함.
- 도구 결과 메시지도 요청과 같은 순서로 쌓임. 짝을 유지하는 것은 **tool_call_id**.
- 도구가 각각 외부 호출이면 소요 시간이 더해짐. 서로 의존하지 않는 호출은 동시에 실행할 수 있음.

동시 실행 버전: 일괄 제출과 일괄 수거

```
with ThreadPoolExecutor() as pool:
    futures = [
        pool.submit(
            TOOLS[c.function.name],
            **json.loads(
                c.function.arguments))
        for c in msg.tool_calls]
    for call, fut in zip(
        msg.tool_calls, futures):
        result = fut.result()
        messages.append(
            tool_msg(call.id, result))
```

[질문] 지금 시각이랑 환불 규정을 같이 알려 주세요.

[동시 실행 결과] get_now

→ 2026-08-25 21:13

[동시 실행 결과] faq_lookup

→ 이용일 전날까지 전액 환불,
당일은 50% 환불입니다.

- 요청을 **전부 제출한 뒤** 결과를 거둬들.
서로 기다리지 않으므로 전체 소요는
가장 느린 도구 하나의 시간에 수렴함.
- 결과가 기록에 붙는 순서는 요청 순서 그대로.
작은 **tool_call_id**가 유지함.

도구 실행 실패와 무중단 완주

실행 예시

[질문] 환불 규정 알려 주세요.

[도구 호출] `faq_lookup({'topic': '환불'})`

[도구 결과] 도구 실행 오류: **KeyError**: '환불'

[최종 답] 환불 규정을 조회하는 중 오류가 발생했습니다. 죄송하지만 현재 정확한 규정을 확인해 드리기 어렵습니다. 관리자나 해당 시설 안내 데스크에 문의해 주세요.

- 조회 항목이 없어 **KeyError**가 났고, 그 에러 문면이 도구 결과로 기록에 들어감.
- 파이썬 예외가 프로그램을 끝내지 않음. 루프는 다음 바퀴로 넘어감.
- 모델은 에러를 읽은 상태에서 다음 행동을 정함. 시스템 프롬프트와 도구 설명에 따라 다른 도구를 다시 호출하거나 답을 지어내지 않게 할 수 있음.

에러를 도구 결과로 되먹이는 처리

```
try:
    result = fn(**args)
except Exception as e:
    result = (f"도구 실행 오류: "
             f"{type(e).__name__}: {e}")
messages.append(
    tool_msg(call.id, result))
```

- 예외를 잡아 **문자열로 바꾸는 것**이 전부.
그 문자열이 평소의 도구 결과 자리에 들어감.
- 실패도 결과이므로 기록의 짝이 깨지지 않음.
요청 하나에 결과 하나가 유지됨.
- 예외 타입과 메시지를 함께 실어야
모델이 무엇이 잘못됐는지 읽을 수 있음.

루프를 멈추는 자리는 최종 답과 반복 한도뿐.
도구 실패는 종료 조건이 아님.

ReAct(Reasoning and Acting): 모델이 생각을 내놓고, 도구를 호출하고, 그 결과를 관찰한 뒤 다시 생각하는 순환.

생각(Thought), 행동(Action), 관찰(Observation)이 대화 기록에 순서대로 쌓임.

생각, 행동, 관찰이 대화 기록에 들어가는 위치

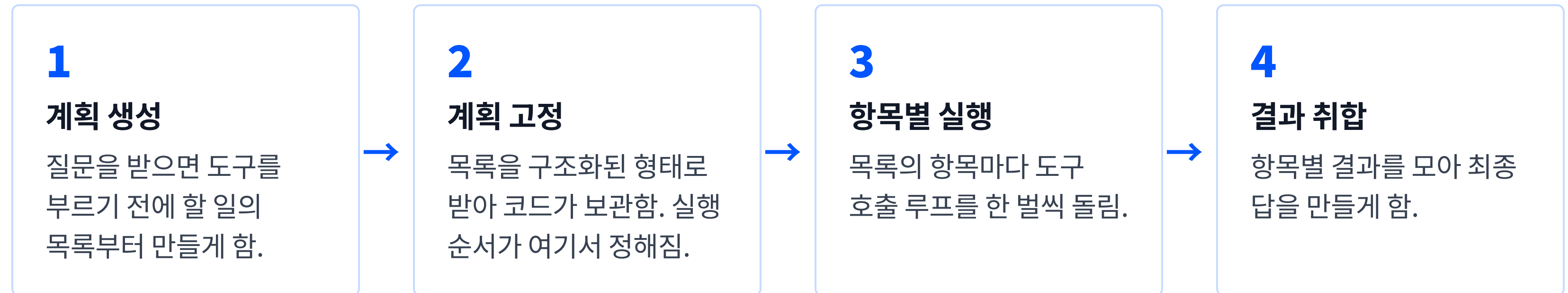
```
msg = res.choices[0].message
if not msg.tool_calls:
    return msg.content
messages.append(msg)
for call in msg.tool_calls:
    fn = TOOLS[call.function.name]
    args = json.loads(
        call.function.arguments)
    messages.append(tool_msg(
        call.id, fn(**args)))
```

- **생각과 행동**: 모델 응답 하나에 판단과 호출 요청이 함께 담겨 기록에 붙음.
- **관찰**: 도구 실행 결과가 role이 tool인 메시지로 붙음.
- 다음 바퀴의 모델 호출에는 이 셋이 전부 실려 나감. 순환은 **기록의 누적**으로 성립함.

추가 부품 없이, 반복문 하나가 이 순환.

Plan-and-Execute: 계획 선행 변형

실행 이전에 할 일의 목록을 먼저 만들고, 그 목록을 차례로 실행하는 방식.



계획 단계가 앞에 하나 더 붙을 뿐, 각 항목의 실행은 같은 도구 호출 루프.

계획 선행 실행의 콘솔 출력

[질문] 환불 규정이랑 주차 안내를
정리해서 알려 주세요.

[계획]

1. 환불 규정의 대상, 기간, 절차 및
예외 사항을 확인합니다.
2. 주차 가능 여부, 위치, 운영 시간,
요금 및 할인 정보를 확인합니다.
3. 확인한 환불 규정과 주차 안내를
이해하기 쉽게 정리해 제공합니다.

[단계 1 실행] `faq_lookup('환불')`

[단계 2 실행] `faq_lookup('주차')`

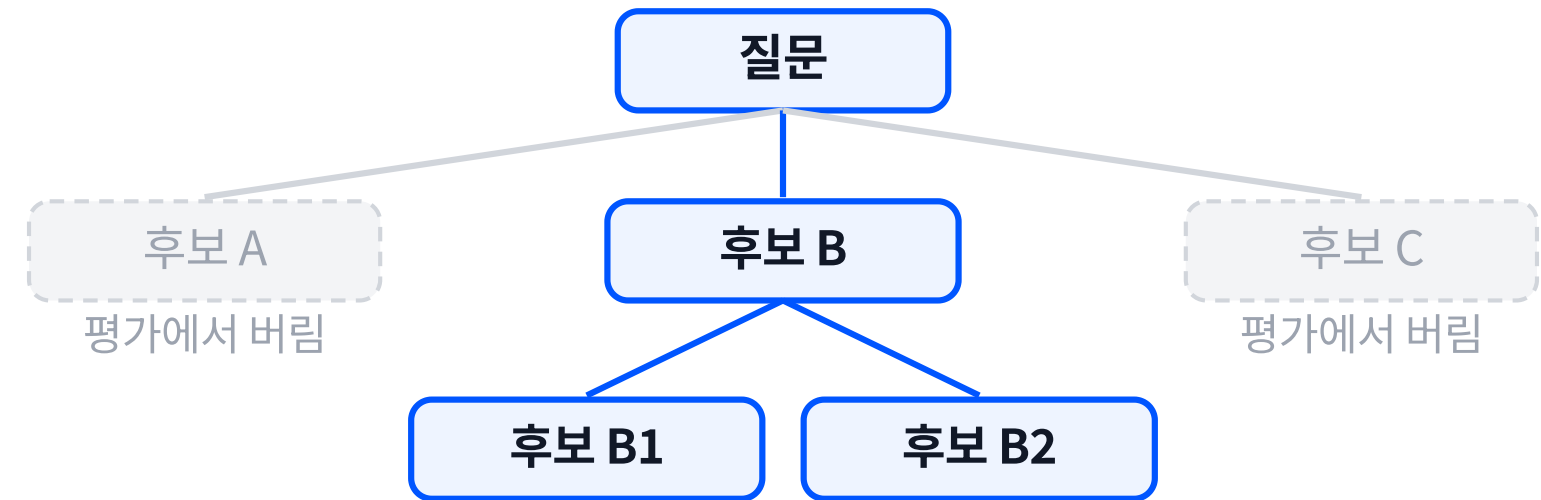
[단계 3 실행] `faq_lookup('환불')` 재 호출

- 도구를 부르기 전에 **할 일 목록**이 먼저 만들어짐.
실행은 그 목록을 위에서부터 소비함.
- 단계 3이 앞 단계가 이미 조회한 항목을 **다시
조회함**.
단계 사이에 맥락이 공유되지 않으면
같은 일이 반복됨. 계획 방식의 비용이
이 자리에서 생김.

ToT (Tree of Thoughts): 후보 가지 탐색

다음 단계 하나를 고르는 대신, 후보 생각(thought) 여러 개를 가지로 펼쳐 두고 각 가지를 평가해 탐색하는 방식.

- 후보 생성, 상태 평가, 탐색(너비 우선 BFS·깊이 우선 DFS)의 세 부분으로 구성됨.
- 평가에서 밀린 가지는 버리고 앞선 분기점으로 되돌아감.
- 탐색 폭과 깊이를 늘릴수록 모델 호출이 곱으로 늘어남.



세 패턴의 대비

비교 항목	ReAct	Plan-and-Execute	ToT
계획 수립 시점	매 바퀴마다 다음 한 수만 정함	실행 이전에 전체 목록을 정함	단계마다 후보 여러 개를 펼침
한 호출이 정하는 것	다음 도구 호출 한 건	할 일 목록 전체	후보 하나의 생성 또는 평가
중간 결과 반영	직전 관찰이 바로 반영됨	다시 세우기 전까지 고정됨	밀린 가지를 버리고 분기점 복귀
모델 호출 횟수	단계 수만큼 늘어남	계획 1회 + 항목별 실행	폭과 깊이의 곱에 비례
맞는 일	다음 수가 이전 결과에 달린 일	단계가 미리 정해지는 일	되돌리기가 필요한 탐색형 일

세 패턴 모두 같은 도구 호출 루프 위에서 돛. 다른 것은 계획을 언제 세우는가, 그리고 하나로 세우는가 여럿으로 나누어 세우는가.

실행 결과를 보고 앞선 판단을 고쳐 다음 수를 정하는 몫을 **자기 성찰(Self-Reflection)**이라 부름. ReAct는 매 바퀴마다, Plan-and-Execute는 계획을 다시 세울 때 이 몫이 작동함.

실습합시다

챕터 요약

- ✓ 에이전트의 실체는 도구 호출 루프. 모델 호출, 도구 호출 여부 판정, 도구 실행, 결과 되먹임의 네 단계가 반복됨
- ✓ 도구는 이름, 설명, 파라미터 스키마를 붙인 함수이고, 설명의 질이 모델의 도구 선택 품질을 결정함
- ✓ 도구 실행 에러는 루프를 멈추지 않음. 에러 메시지를 도구 결과로 되먹여 모델이 그 상태에서 답을 만듦
- ✓ ReAct는 생각·행동·관찰이 대화 기록에 쌓이며 도는 루프, Plan-and-Execute는 실행 이전에 할 일 목록을 먼저 세우는 변형, ToT는 후보 생각을 가지로 펼쳐 평가하며 탐색하는 변형

Agent 개요

랭체인 기초: 부품과 체인

부품 지도 / 체인 조립 / 실행 방식

학습 목표

- 손으로 짠 호출 코드의 어느 부분(모델 초기화·프롬프트 조립·출력 파싱)이 랭체인(LangChain)의 어느 부품으로 흡수되는지, 두 코드를 대조해 짚을 수 있습니다.
- `init_chat_model` 모델, 프롬프트 템플릿, 구조화 출력(Structured Output)을 `|`로 조립한 체인을 작성하고 `invoke`·`stream` 두 방식으로 실행할 수 있습니다.
- 손이 맡던 출력 형식 강제, 파싱, 필드 검증이 구조화 출력(Structured Output)으로 접힌 까닭을, 체인과 프롬프트 템플릿과 Output Parser가 랭체인 1.0에서 놓인 자리와 짝지어 설명할 수 있습니다.

✓ 에이전트 동작 원리

▶ 랭체인 기초

RAG: 검색기 구축

RAG 에이전트 서비스

RAG와 LLM wiki

손 코드 1: 지시문 조립과 재시도 진입

```
system = (
    "너는 시설 안내 담당자다. "
    "아래 FAQ만 근거로 답한다.\n"
    f"FAQ: {FAQ_CONTEXT}\n"
    "반드시 JSON 하나만 출력한다. "
    "다른 문장, 인사말, 코드 블록 금지.\n"
    '형식: {"topic": "<주제>", '
    '"answer": "<답변>"}'
)
last_error = None
for attempt in range(3):
    user = QUESTION
    if last_error:
        user = (
            f"{QUESTION}\n직전 응답이 "
            f"형식 오류였다: {last_error}\n"
            "형식을 지켜 다시 출력하라.")
```

한 번의 질의를 보내기 전에 손으로 쓰는 것

손이 맡는 일	줄 수
역할 지시·근거 문서·출력 형식 지시 조립	9
재시도 반복문과 직전 오류 되먹임	8

출력 형식은 문장으로만 강제됨. 모델이 그 문장을 지킬지는 **응답을 받아 보기 전까지 알 수 없음.**

손 코드 2: 응답 파싱과 필드 검증

```
res = completion(
    model="openai/gpt-5.6-luna",
    messages=[{"role": "system",
               "content": system},
              {"role": "user",
               "content": user}])
text = res.choices[0].message.content
text = text.strip()
if text.startswith("`"):
    text = text.strip("`")
    text = text[text.find("{"):]
try:
    data = json.loads(text)
    return FaqAnswer(**data), attempt + 1
except json.JSONDecodeError as e:
    last_error = f"JSON 파싱 실패: {e}"
except ValidationError as e:
    last_error = f"필드 검증 실패: {e}"
```

응답 문자열이 자료 구조가 되기까지

손이 맡는 일	줄 수
코드 펜스 제거	3
JSON 파싱과 예외 처리	4
필드 검증과 실패 처리	4

- pydantic이 맡는 몫은 **필드 검증**까지.
JSON 형식 강제와 dict 변환은 손에 남음.

코드의 대부분은 지시문 조립부와 파싱·검증부. 그중
질문 하나를 다루는 줄은 **두 줄**이고, 나머지는 **형식을
지키게 만드는 줄.**

같은 질문, 같은 답

손으로 다 하는 버전

- 지시문 조립
- 출력 형식 문장 강제
- 코드 펜스 제거
- JSON 파싱
- 필드 검증
- 재시도 3회

부품으로 조립한 버전

```
llm = init_chat_model(  
    "gpt-5.6-luna",  
    model_provider="litellm")  
chain = (prompt  
    | llm.with_structured_output(FaqAnswer))
```

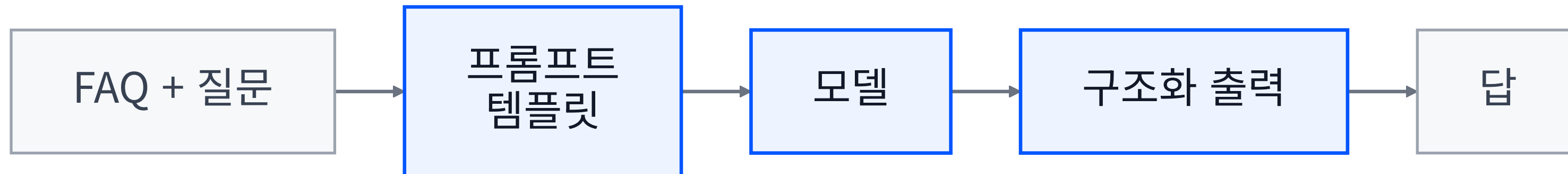
두 버전의 실행 결과 (콘솔 출력)

```
[손 버전 | 시도 1회] topic=운영시간  
| answer=매일 09:30부터 21:00까지  
운영합니다.  
[체인 버전] topic=운영시간  
| answer=매일 09:30부터 21:00까지  
운영합니다.
```

부품 셋을 파이프로 잇는 안내 체인

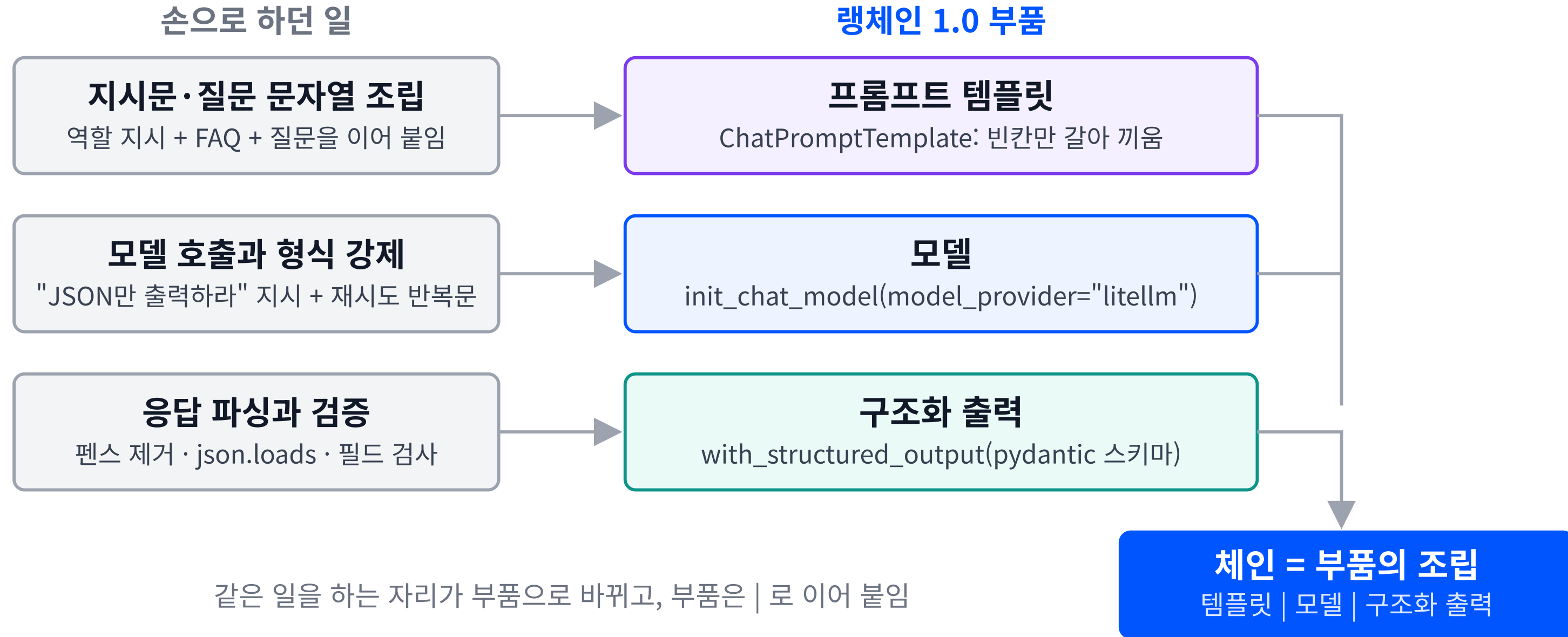
문제 구름월드 안내 프로그램을 맡은 개발자예요. 지시문을 조립하고 답을 정해진 양식으로 받아 내는 코드를 손으로 한 줄씩 짜다 보니 매번 길어져요.

목표 자주 쓰는 부품 세 개(지시문 틀·모델·답 양식)를 끼워 맞추는 방식으로 같은 안내 프로그램을 짧게 다시 만들어봅시다.



- **프롬프트 템플릿**: FAQ와 질문을 받아 → 정해진 역할 지시문의 빈칸에 끼워 → 메시지 묶음을 넘긴다
- **모델**: 고정 지침(시스템 프롬프트)이 채워진 메시지 묶음을 입력으로 받아 → 한 번 불러 → 응답을 넘긴다
- **구조화 출력**: 응답을 받아 → 정해진 형식대로 칸을 채우게 해 → 항목과 답을 넘긴다

랭체인 1.0 부품 지도



체인과 프롬프트 템플릿

체인 (Chain)

"부품 여러 개를 정해진 순서로 이어 붙인 실행 단위"

앞 부품의 출력이 뒤 부품의 입력이 되도록 묶어 두고, 바깥에서는 한 번의 호출로 실행하는 구조.

랭체인 1.0에서의 자리: 조립 연산자 `|` 로 이어 붙인 것이 곧 체인. 전용 클래스(`LLMChain` 등)는 별도 패키지 `langchain-classic`으로 이동함.

프롬프트 템플릿 (Prompt Template)

"고정 문장 + 값이 바뀌는 빈칸"

역할 지시와 문장 골격을 고정해 두고, 매번 달라지는 값만 중괄호 빈칸으로 남긴 메시지 묶음.

랭체인 1.0에서의 자리:

`ChatPromptTemplate`으로 존속함.

`system`·`user` 역할별로 선언하고, 빈칸은 실행 시점에 채워짐.

Output Parser와 LCEL

Output Parser

"응답 문자열을 정해진 자료 구조로 바꾸는 부품"

모델이 돌려준 문자열을 파싱해 자료 구조로 만들고, 형식이 어긋나면 실패로 처리하는 부품.

랭체인 1.0에서의 자리: 필드 이름과 자료형을 선언한 pydantic **BaseModel** 클래스를 모델에 직접 물리는 구조화 출력 (**with_structured_output**)이 표준. 파서를 따로 붙이던 자리가 스키마 하나로 접힘.

LCEL (LangChain Expression Language)

"부품을 |로 잇는 조립 표기"

실행 순서를 코드의 배치 순서로 그대로 적는 표기법. 왼쪽 부품의 출력이 오른쪽 부품의 입력이 됨.

랭체인 1.0에서의 자리: 현행 문법. **langchain-core**의 조립 연산자 그대로이며, 실습 체인도 이 표기로 조립함. 이전 세대로 물러난 것은 **LLMChain**류 전용 클래스뿐.

네 부품의 이전 자리와 현재 자리

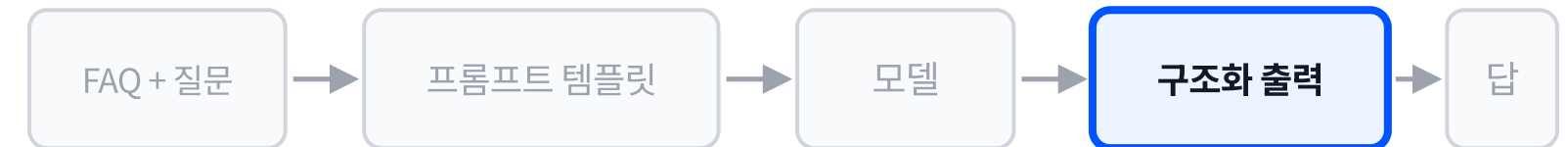
부품	정의 요지	랭체인 1.0에서의 자리
체인 (Chain)	부품을 정해진 순서로 이어 붙인 실행 단위	조립 연산자 <code> </code> 로 이어 붙인 것이 체인. <code>LLMChain</code> 류 전용 클래스만 <code>langchain-classic</code> 으로 이동
프롬프트 템플릿	고정 문장 + 값이 바뀌는 빈칸	<code>ChatPromptTemplate</code> 으로 존속
Output Parser	응답 문자열을 자료 구조로 변환·검증	pydantic 스키마를 모델에 직접 물리는 구조화 출력으로 대체
LCEL	부품을 <code> </code> 로 잇는 조립 표기	현행 문법. <code>langchain-core</code> 의 조립 연산자 그대로이며 실습 체인이 이 표기로 조립됨

1단계: 출력 스키마 선언

```
class FaqAnswer(BaseModel):
    topic: str
    answer: str

llm = init_chat_model(
    "gpt-5.6-luna", model_provider="litellm")

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 시설 안내 담당자다. 아래 FAQ만 근거로 "
     "답한다.\nFAQ: {faq}"),
    ("user", "{question}"),
])
chain = (prompt
        | llm.with_structured_output(FaqAnswer))
out = chain.invoke({"faq": FAQ_CONTEXT,
                   "question": "몇 시까지 여나요?"})
print(out.topic, out.answer)
```



BaseModel을 상속한 클래스 하나가 받을 답의 규격. 필드 이름과 자료형이 그 규격의 전부.

이 규격이 모델에 물려 나가므로, 형식을 지키라는 문장을 따로 쓰지 않음.

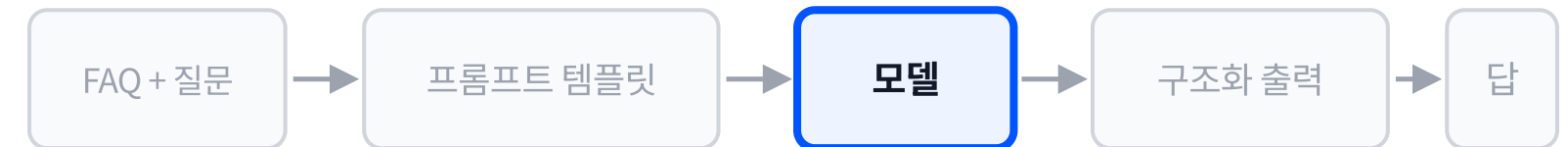
필드를 늘리거나 이름을 바꾸면 돌아오는 자료 구조가 그대로 따라 바뀜.

2단계: 모델 초기화

```
class FaqAnswer(BaseModel):
    topic: str
    answer: str

llm = init_chat_model(
    "gpt-5.6-luna", model_provider="litellm")

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 시설 안내 담당자다. 아래 FAQ만 근거로 "
     "답한다.\nFAQ: {faq}"),
    ("user", "{question}"),
])
chain = (prompt
        | llm.with_structured_output(FaqAnswer))
out = chain.invoke({"faq": FAQ_CONTEXT,
                   "question": "몇 시까지 여나요?"})
print(out.topic, out.answer)
```



init_chat_model은 모델 이름과 제공자를 받아 호출 가능한 모델 객체를 돌려줌.

model_provider가 어느 경로로 호출할지를 정함. 모델 이름만 바꾸면 나머지 코드는 그대로.

이 객체가 체인의 가운데 부품이 됨.

3단계: 프롬프트 템플릿 선언

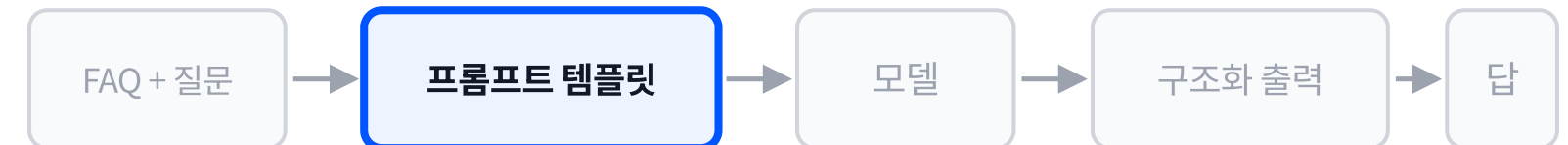
```
class FaqAnswer(BaseModel):
    topic: str
    answer: str

llm = init_chat_model(
    "gpt-5.6-luna", model_provider="litellm")

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 시설 안내 담당자다. 아래 FAQ만 근거로 "
     "답한다.\nFAQ: {faq}"),
    ("user", "{question}"),
])

chain = (prompt
        | llm.with_structured_output(FaqAnswer))
out = chain.invoke({"faq": FAQ_CONTEXT,
                   "question": "몇 시까지 여나요?"})

print(out.topic, out.answer)
```



역할별 메시지를 목록으로 선언함. **system**은 지시, **user**는 질문 자리.

중괄호 **{faq}** · **{question}**이 실행 시점에 값으로 채워지는 빈칸.

여러 줄 문자열 연결 대신, 이 선언 하나로 지시문과 질문 자리가 고정됨.

4단계: 조립과 실행

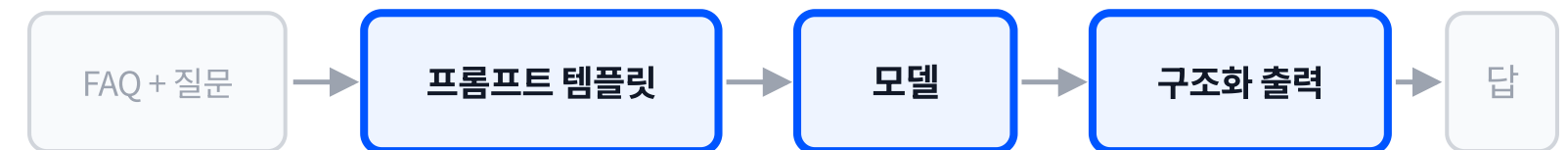
```
class FaqAnswer(BaseModel):
    topic: str
    answer: str

llm = init_chat_model(
    "gpt-5.6-luna", model_provider="litellm")

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 시설 안내 담당자다. 아래 FAQ만 근거로 "
     "답한다.\nFAQ: {faq}"),
    ("user", "{question}"),
])

chain = (prompt
        | llm.with_structured_output(FaqAnswer))
out = chain.invoke({"faq": FAQ_CONTEXT,
                   "question": "몇 시까지 여나요?"})

print(out.topic, out.answer)
```



|가 왼쪽 부품의 출력을 오른쪽 부품의 입력으로 넘김. 이어 붙인 결과가 체인.

invoke에 넘기는 딕셔너리의 키가 템플릿의 빈칸 이름과 짝을 이룸.

돌아오는 값은 문자열이 아니라 **FaqAnswer** 객체이므로 **out.topic**처럼 필드로 꺼냄.

조각으로 받는 실행: stream

```
llm = init_chat_model(
    "gpt-5.6-luna", model_provider="litellm",
    streaming=True)
prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 시설 안내 담당자다. "
               "FAQ: {faq}"),
    ("user", "{question}"),
])
chain = prompt | llm
n = 0
for c in chain.stream({"faq": FAQ_CONTEXT,
                      "question": "환불 규정을 자세히 "
                                   "설명해 주세요."}):
    if c.content:
        n += 1
        print(c.content, end="", flush=True)
print(f"\n[stream] 청크 수 = {n}")
```

streaming=True가 없으면 응답이 조각으로 나뉘지 않고 한 덩어리로 도착함. 실행 방식만 바꾼다고 조각이 오지는 않음.

stream은 조각을 하나씩 내주는 반복 가능한 값을 돌려줌. 마지막까지 모으면 **invoke**의 결과와 같은 내용.

[stream] 청크 수 = 117

체인에 함수 끼워 넣기

```
def to_notice(out: FaqAnswer) -> str:
    return (f"[{out.topic}] 안내: "
            f"{out.answer}")

chain = (prompt
        | llm.with_structured_output(FaqAnswer)
        | to_notice)
print(chain.invoke(
    {"faq": FAQ_CONTEXT,
     "question": QUESTION}))
```

[운영시간] 안내: 매일 09:30부터
21:00까지 운영합니다.

- 부품의 자격은 **입력 하나를 받아 값 하나를 돌려주는 것**뿐. 파이썬 함수도 그 자격을 갖추므로 **|**로 이어 붙일 수 있음.
- 앞 부품의 출력(**FaqAnswer** 객체)이 함수의 인자가 되고, 함수의 반환값이 체인의 결과가 됨.

끼워 넣는 자리	함수가 하는 일
템플릿 앞	질문 문자열 다듬기
모델 뒤	응답에서 필요한 부분만 꺼내기
체인 끝	뒤 단계가 받을 형태로 변환

같은 체인의 세 가지 실행

실행 방식	돌려받는 결과의 모양	쓰는 상황
invoke	완성된 결과 하나	결과 전체가 있어야 다음 일을 할 때
stream	조각의 연속	답이 만들어지는 동안 화면에 흘러 보낼 때
ainvoke	완성된 결과 하나(비동기, async)	여러 호출을 동시에 걸어 두고 기다릴 때

체인은 그대로 두고 부르는 방법만 바꿈. 조립한 부품과 순서는 세 방식에서 동일함.

같은 손 루프, 랭체인 문법으로

```
@tool(parse_docstring=True)
def faq_lookup(topic: str) -> str:
    """시설 FAQ에서 항목을 조회한다.
    운영시간, 주차, 환불 질문에 쓴다. [...]"""
    return FAQ[topic]

llm = init_chat_model("gpt-5.6-luna",
    model_provider="litellm")
llm_tools = llm.bind_tools(
    [faq_lookup, get_now])
```

4-1에서 손으로 쓴 것	랭체인 문법
도구 설명 JSON 스키마 27줄	@tool 데코레이터(독스트링이 설명이 됨)
도구 이름과 함수를 짜지은 표	bind_tools([faq_lookup, get_now])
completion(model, messages, tools=)	llm_tools.invoke(messages)
딕셔너리 메시지 {"role": ...}	메시지 객체 HumanMessage AIMessage · ToolMessage

도구 스키마와 등록이 이 두 줄(@tool·bind_tools)이 됨. 도구 호출 반복문은 그대로 손에 남음.

모델이 도구를 부르는 순간: tool_calls

[질문] 지금 몇 시야?

[tool_calls] `get_now {}`

[최종 답] 지금은 **2026년 8월 28일
오후 8시 21분**입니다.

[질문] 환불 규정 알려 줘

[tool_calls] `faq_lookup`

`{'topic': '환불'}`

[최종 답] 환불 규정은 다음과 같습니다.

- **이용일 전날까지:** 전액 환불
- **이용 당일:** 50% 환불

- 모델의 응답은 **AIMessage**로 돌아오고, 그 안의 **tool_calls**에 도구 이름과 인자가 담김.
- 인자가 처음부터 딕셔너리라서 4-1에서 하던 **json.loads** 문자열 파싱이 필요 없음.
- **tool_calls**가 비어 있으면 최종 답. 이 판정 위치는 4-1 손 루프와 동일함.
- 도구 실행은 **faq_lookup.invoke(call["args"])**. 도구도 부품이므로 invoke로 부름.

실습합시다

챕터 요약

- ✓ 지시문 조립, 출력 형식 강제, 파싱, 검증, 재시도를 손으로 쓰면 코드가 길어지고, 프레임워크는 그 자리를 부품으로 제공함
- ✓ 랭체인 1.0의 기본 부품은 모델 초기화(`init_chat_model`)·프롬프트 템플릿·구조화 출력이고, 조립 연산자 `|`로 이어 붙인 것이 체인
- ✓ 손이 말던 출력 형식 강제, 파싱, 필드 검증이 구조화 출력으로 접히면서, 체인·Output Parser가 가리키던 자리는 조립 연산자 `|`와 스키마 선언으로 정리되고, LCEL은 그 조립 표기로 존속함
- ✓ 같은 체인을 `invoke`로 한 번에, `stream`으로 조각내어 실행할 수 있고, `stream`은 `streaming=True` 선언을 함께 요구함

Agent 개요

RAG: 검색기 구축

검색기(리트리버, retriever)가 필요한 이유 / 구축과 질의 파이프라인 / 검색 품질

학습 목표

- RAG의 다섯 단계(Load, Split, Embed, Store, Retrieve)를 각 단계의 입력과 산출물을 들어 설명할 수 있습니다.
- 문서를 임베딩해 Chroma에 저장하고, 질문으로 유사 조각과 유사도 점수를 검색하는 코드를 작성할 수 있습니다.
- 유사도 임계값으로 저품질 검색을 걸러내는 기준을 실제 측정한 점수로 정하고, 걸러진 질문을 보낼 경로를 설계할 수 있습니다.

✓ 에이전트 동작 원리

✓ 랭체인 기초

▶ **RAG: 검색기 구축**

RAG 에이전트 서비스

RAG와 LLM wiki

문서에만 있는 규정을 물었을 때의 맨 호출 응답

[질문] 구름월드 자유이용권을 당일에 환불하면 수수료가 얼마인가요?

[맨 호출 응답] 구름월드 자유이용권의 당일 환불 수수료는 ****구매처와 상품 약관에 따라 달라질 수 있습니다.**** 당일에는 보통 ****환불 불가****이거나 ****결제금액의 10%가 공제****되는 경우가 많습니다.

정확한 금액을 확인하려면 구매한 예매처의 ****취소·환불 규정****을 확인하거나, 예매번호와 함께 구름월드 고객센터에 문의해 주세요.

모델 파라미터에는 이 시설의 규정이 없음. 근거 문서 없이 만들어진 문장은 지금 참인지 보증할 수 없음.

문제상황과 목표

문제

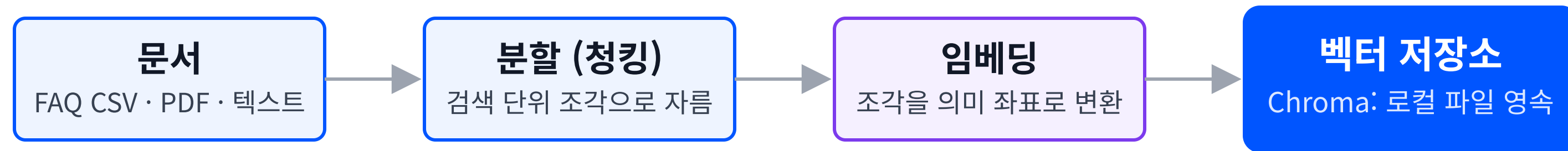
구름월드 고객센터예요. FAQ가 32개인데 손님 질문은 매번 표현이 달라서, 어느 FAQ 항목이 답인지 찾아 주는 게 일이에요.

목표

FAQ를 미리 정리해 두었다가, 질문이 오면 표현이 달라도 가장 가까운 FAQ 항목을 찾아 주는 검색기를 만들어봅시다.

RAG 파이프라인: 구축과 질의

구축 (한 번)



질의 (매번)



구축은 한 번, 질의는 매번. 두 레인이 벡터 저장소에서 만남

구축은 문서가 바뀔 때 한 번, 질의는 질문이 들어올 때마다 실행됨.

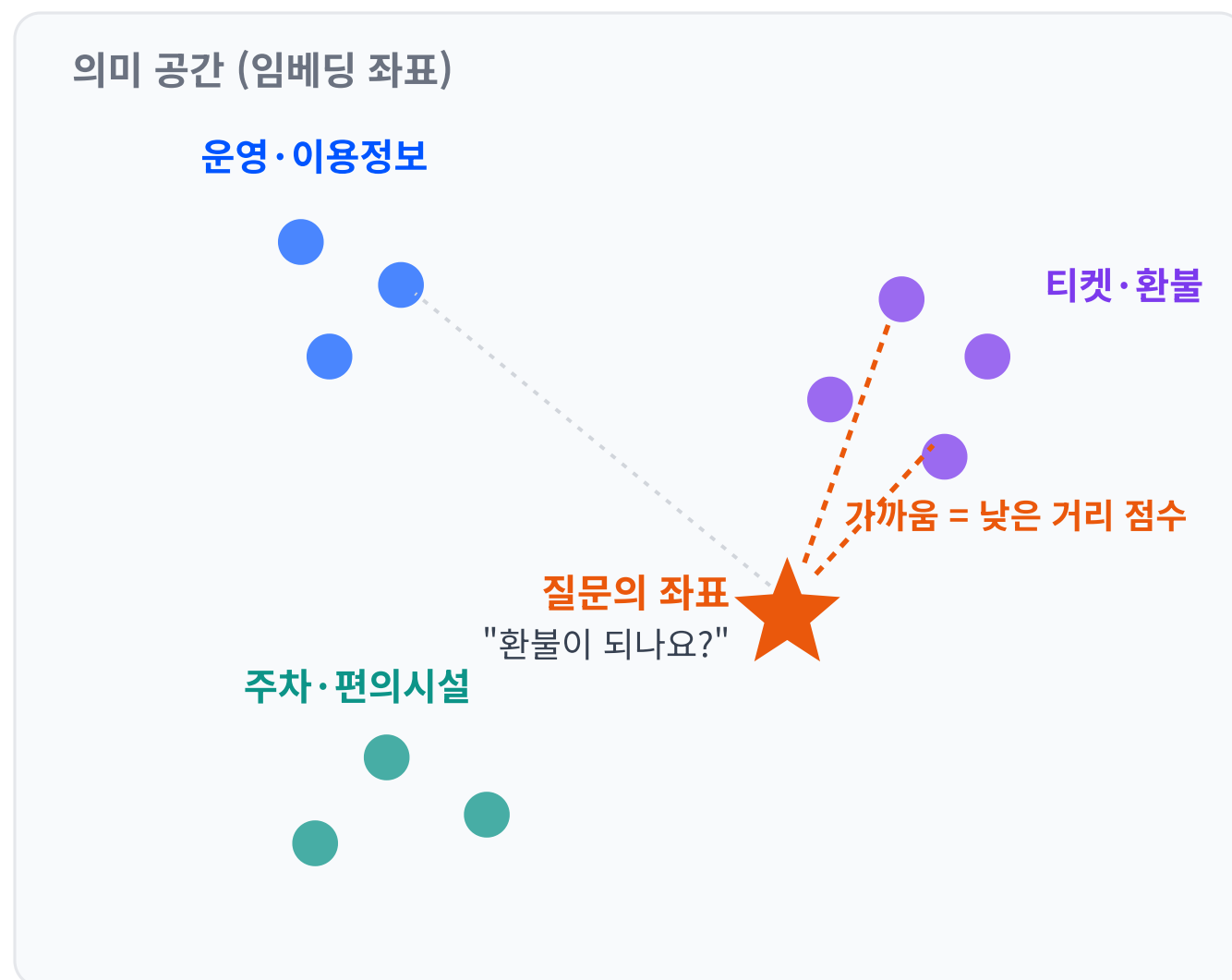
각 단계별 요구사항

단계	무엇을 받아	어떻게	무엇을 넘긴다
문서	FAQ CSV를	빈 행을 버리고	문서 목록을 넘긴다
분할 (칭킹)	문서를	검색 단위 조각으로 잘라	조각을 넘긴다
임베딩	조각을	의미 좌표로 바꿔	벡터를 넘긴다
벡터 저장소	벡터와 조각을	로컬 파일에 저장해 두고	조회 요청에 가까운 조각을 돌려준다
질문	질문 한 줄을	그대로	질문 임베딩에 넘긴다
질문 임베딩	질문을	조각과 같은 좌표계로 바꿔	벡터를 넘긴다
유사도 검색	질문 벡터를	저장소에서 가까운 순으로 조회해	조각과 거리 점수를 넘긴다
상위 조각	조각과 점수를	상위 몇 개만 골라	프롬프트에 동봉한다
답	조각이 동봉된 질문을	그 근거로만 답을 만들어	답을 넘긴다

RAG(Retrieval-Augmented Generation)는 질문과 가까운 문서 조각을 **먼저 검색해 프롬프트에 함께 넣고** 답하게 하는 방식.

모델을 다시 학습시키지 않고, 답의 근거를 그때그때 공급하는 경로.
검색이 실패하면 근거 없는 답이 그대로 나오므로 검색의 품질이 답의 품질.

임베딩과 유사도 검색



질문과 가까운 조각이 검색되고, 거리 점수가 그 "가까움"의 숫자

- **임베딩(Embedding)**: 텍스트를 의미 좌표인 숫자 벡터로 바꾸는 변환.
text-embedding-3-small의 출력은 1536차원 벡터.
- **유사도 검색(Similarity Search)**: 질문 벡터와 가까운 조각을 고르는 연산.
반환값에 거리(distance) 점수가 함께 옴.
- 거리 점수는 **작을수록 가까움**.
- 조각과 질문이 같은 임베딩 모델을 거쳐야 같은 좌표계에 놓고 거리 비교가 성립함.

청킹: 검색 단위로 자르는 분할

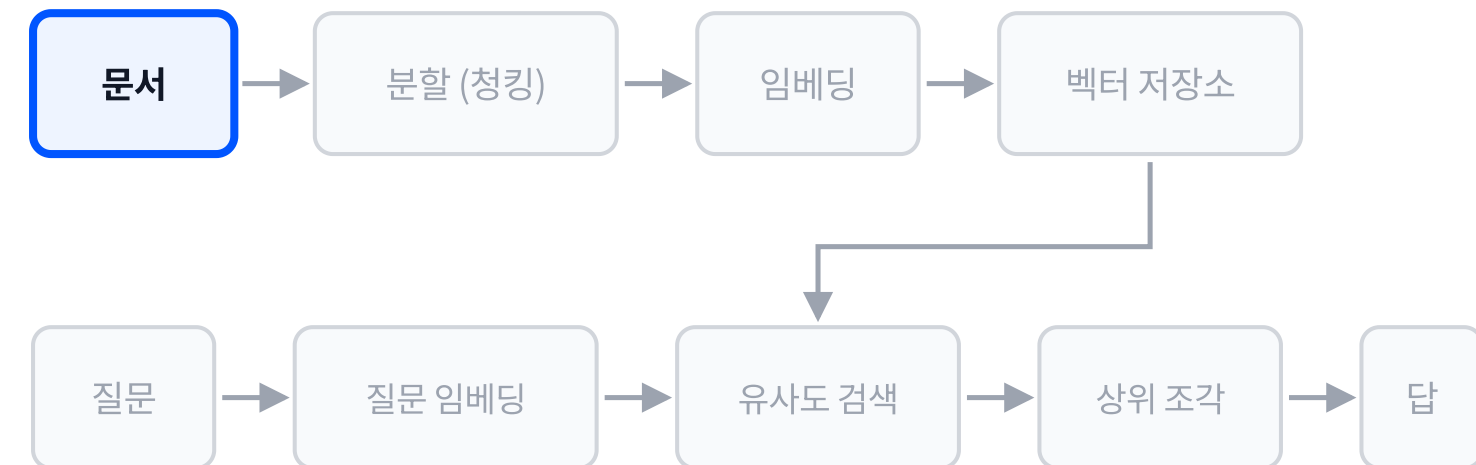
- **청킹(Chunking)**: 문서를 검색 단위 조각으로 자르는 분할.
- 자르는 이유는 **Context Window** 제약.
모델이 한 번에 받을 수 있는 **토큰(Token)** 수가 정해져 있어 문서 전체를 매번 넣을 수 없음.
- 토큰은 모델이 글을 세는 단위이고,
입력과 출력이 같은 창을 나눠 씀.
- 조각이 크면 관계없는 문장이 함께 딸려 오고,
작으면 답에 필요한 문맥이 잘림.

파라미터	기본값	하는 일
chunk_size	1000자	조각 하나의 최대 길이
chunk_overlap	200자	이웃 조각과 겹치는 길이

FAQ CSV는 한 행이 질문과 답 한 쌍이라 행 단위가 그대로 검색 단위. 구름월드 FAQ 32행은 별도 분할 없이 조각 32개로 적재됨.

1단계: FAQ 파일을 한 행씩 읽기

```
import csv
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
docs = []
with open(CSV_PATH, encoding="utf-8-sig",
          newline="") as f:
    for i, row in enumerate(csv.DictReader(f), 1):
        text = (f"[{row['Category']}] "
                f"Q: {row['Question']}\n"
                f"A: {row['Answer']}")
        docs.append(Document(page_content=text,
                              metadata={"row": i,
                                         "category": row["Category"]}))
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")
db = Chroma.from_documents(docs, embeddings,
                           persist_directory=DB_DIR)
```



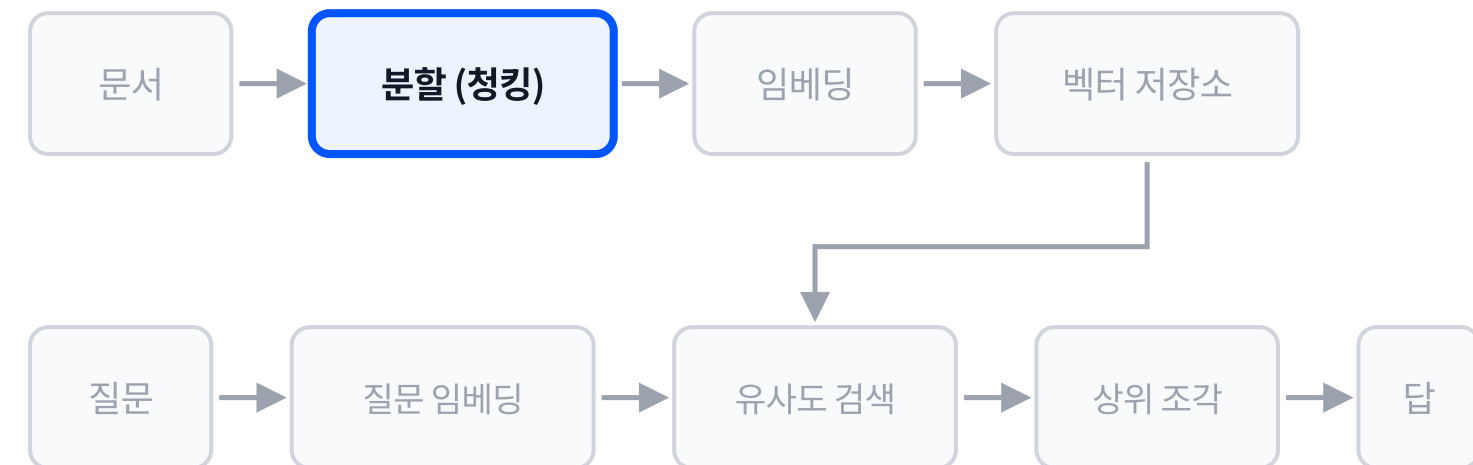
csv.DictReader 첫 줄을 열 이름으로 삼아 각 행을 딕셔너리로 돌려줌.

utf-8-sig 파일 앞머리에 붙는 BOM 표시를 함께 걷어내는 인코딩 지정. 엑셀에서 저장한 CSV의 첫 열 이름이 깨지는 원인이 이것.

docs 리스트 하나가 적재 결과 전체.

2단계: 행을 문서 조각으로 구성

```
import csv
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
docs = []
with open(CSV_PATH, encoding="utf-8-sig",
          newline="") as f:
    for i, row in enumerate(csv.DictReader(f), 1):
        text = (f"[{row['Category']}] "
                f"Q: {row['Question']}\n"
                f"A: {row['Answer']}")
        docs.append(Document(page_content=text,
                              metadata={"row": i,
                                         "category": row["Category"]}))
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")
db = Chroma.from_documents(docs, embeddings,
                           persist_directory=DB_DIR)
```



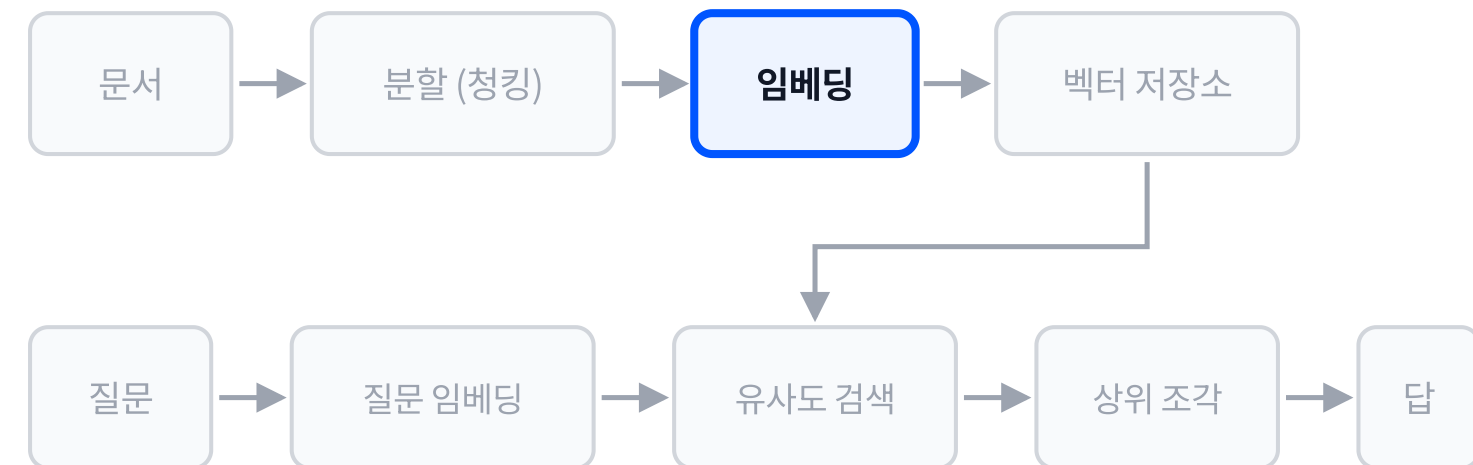
Document 검색 대상 본문(page_content)과 부가 정보(metadata) 두 칸을 가진 객체. 벡터 저장소가 받는 단위가 이것.

page_content 분류·질문·답을 한 덩어리 문자열로 합쳐 조각의 본문으로 삼음. 검색되는 대상은 이 문자열.

metadata 행 번호와 분류를 함께 실어 두면 검색된 조각이 문서 어디에서 왔는지 되짚을 수 있음.

3단계: 임베딩 모델 지정

```
import csv
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
docs = []
with open(CSV_PATH, encoding="utf-8-sig",
          newline="") as f:
    for i, row in enumerate(csv.DictReader(f), 1):
        text = (f"[{row['Category']}]"
                f"Q: {row['Question']}\n"
                f"A: {row['Answer']}")
        docs.append(Document(page_content=text,
                             metadata={"row": i,
                                       "category": row["Category"]}))
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")
db = Chroma.from_documents(docs, embeddings,
                           persist_directory=DB_DIR)
```



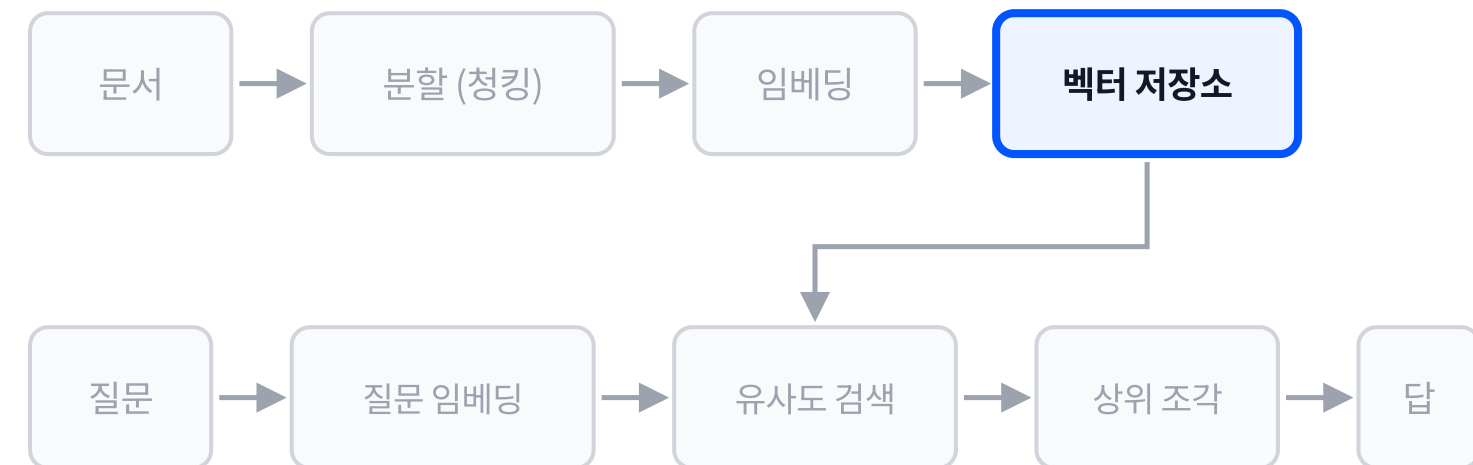
OpenAIEmbeddings 텍스트를 벡터로 바꾸는 변환기. 모델 이름이 좌표계와 차원 수를 정함.

이 줄에서는 변환기만 준비되고 호출은 아직 일어나지 않음.

조각을 넣을 때와 질문을 검색할 때 같은 변환기를 써야 거리 비교가 성립함. 모델을 바꾸면 저장소를 다시 만들어야 함.

4단계: 벡터 저장소에 영속 저장

```
import csv
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
docs = []
with open(CSV_PATH, encoding="utf-8-sig",
          newline="") as f:
    for i, row in enumerate(csv.DictReader(f), 1):
        text = (f"[{row['Category']}] "
               f"Q: {row['Question']}\n"
               f"A: {row['Answer']}")
        docs.append(Document(page_content=text,
                              metadata={"row": i,
                                         "category": row["Category"]}))
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")
db = Chroma.from_documents(docs, embeddings,
                           persist_directory=DB_DIR)
```



Chroma 벡터와 원문 조각을 함께 담은 벡터 저장소(Vector Store).

from_documents 조각 전부를 임베딩해 저장소에 넣는 호출. 임베딩 호출이 실제로 일어나는 자리가 여기.

persist_directory 지정한 폴더에 파일로 남음. 다음 실행에서는 다시 만들지 않고 열어서 쓰기만 함.

적재부 교체: CSV에서 PDF로

CSV 적재

"행 하나가 조각 하나"

DictReader로 행을 읽어 Document를 만듦.
항목이 이미 나뉘어 있어 분할이 필요 없음.



PDF 적재

"쪽에서 뽑아 잘라 쓴다"

pymupdf로 쪽마다 텍스트를 추출해 잇고,
청킹으로 조각을 만든 뒤 같은 Document로
구성함.

바뀌는 것은 적재부 한 곳. 임베딩, 저장, 검색 세 단계는 그대로.

점수와 함께 돌아오는 상위 조각

```
db = Chroma(persist_directory=DB_DIR,
            embedding_function=embeddings)
q = "자유이용권 환불이 되나요?"
hits = db.similarity_search_with_score(
    q, k=3)
for d, s in hits:
    print(f"score={s:.4f} | {d.page_content}")
```

반환값은 조각과 거리 점수의 쌍.

```
score=0.6878 | [티켓] Q: 자유이용권 환불
규정이 어떻게 되나요? / A: 이용일
전날까지 취소하면 전액 환불되고 ...
score=0.9641 | [티켓] Q: 자유이용권
종류가 어떻게 되나요? ...
score=1.0076 | [티켓] Q: 연간이용권
혜택이 궁금합니다. ...
```

이름	뜻
k	가져올 조각 수
score	질문과의 거리. 작을수록 가까움

벡터 저장소 내부 열람

```
got = db.get(limit=2)
print(len(db.get()["ids"]))
print(got["documents"][0][:50])
print(got["metadatas"][0])
```

저장소가 무엇을 들고 있는지 직접 확인함.

```
32
[이용정보] Q: 구름월드 운영시간이
어떻게 되나요? / A: 구름월드는 매일
09:30~21...
{'row': 1, 'category': '이용정보'}
```

칸	담긴 것
ids	조각마다 붙은 식별자
documents	조각의 본문 문자열
metadatas	적재할 때 실어 둔 부가 정보

임계값 컷: 문서 밖 질문 걸러내기

```
THRESHOLD = 1.5
d, s = db.similarity_search_with_score(
    q, k=1)[0]
verdict = "통과" if s <= THRESHOLD else "컷"
print(f"[질문] {q}")
print(f" 1위 score={s:.4f} → {verdict}")
```

임계값(threshold)은 문서 안 질문의 점수 분포와
문서 밖 질문의 점수 분포 사이에 낫는 선.

적재 문서 수: 32
[임계값] 자유이용권 환불이 되나요?
→ 1위 score=0.6878 통과
[임계값] 파이썬 리스트 정렬은?
→ 1위 score=1.6803 컷 (임계값 1.5)

- 검색은 **언제나 무언가를 반환함**.
문서에 답이 없어도 가장 가까운 조각이 나옴.
- 점수를 보지 않으면 관계없는 조각이
그대로 근거로 쓰임.

컷된 질문을 보낼 곳

경로	언제 고르나	함께 정할 것
모른다고 답하기	이 문서가 유일한 근거일 때	사과 문구와 안내처
질의를 바꿔 재검색	표현이 문서와 달라 못 찾았다고 볼 때	재시도 횟수 상한
웹 검색으로 폴백(fallback)	문서 밖 지식이 답인 질문일 때	출처 표기와 신뢰 수준

임계값은 서비스가 무엇을 근거로 답하는지 정하는 선. 컷 다음에 갈 곳까지 정해야 설계가 닫힘.

실습합시다

챕터 요약

- ✓ RAG는 질문과 가까운 문서 조각을 검색해 프롬프트에 공급하는 방식이고, 구축은 Load, Split, Embed, Store, Retrieve 다섯 단계로 이루어짐
- ✓ 임베딩은 텍스트를 의미 좌표로 바꾸는 변환이고, 유사도 검색은 그 좌표 공간에서 질문과 가까운 조각을 고르는 연산
- ✓ 문서를 조각으로 자르는 이유는 Context Window와 토큰 제약이며, 조각 크기와 겹침이 검색 품질을 좌우함
- ✓ 벡터 저장소는 폴더에 파일로 남아, 재실행 시 다시 만들지 않고 열어서 검색만 함
- ✓ 검색 결과에는 거리 점수가 따라오고, 임계값으로 저품질 검색을 걸러낸 뒤 갈 곳을 정해야 검색 설계가 닫힘

Agent 개요

RAG 에이전트 서비스

도구화 / 검색 판단 / 교정 재검색 / 서비스 노출

학습 목표

- 검색기를 도구로 등록한 에이전트가 질문에 따라 검색 여부를 스스로 판단하는 동작을, 검색이 일어난 질문과 일어나지 않은 질문의 실행 트레이스로 설명할 수 있습니다.
- 검색 품질이 낮을 때 질의를 고쳐 재검색하는 교정 분기를 재시도 상한과 함께 손 루프에 추가할 수 있습니다.
- RAG 에이전트를 FastAPI 엔드포인트로 노출하고 curl 호출로 JSON 응답을 확인할 수 있습니다.

✓ 에이전트 동작 원리

✓ 랭체인 기초

✓ RAG: 검색기 구축

▶ **RAG 에이전트 서비스**

RAG와 LLM wiki

이 강의에서 쓰는 FastAPI 최소 지식

라우터 함수 (route function)

주소 하나에 파이썬 함수 하나를 짝지어 등록하는 장식자. 그 주소로 요청이 오면 그 함수가 실행됨.

```
@app.post("/ask")
```

JSON 응답 (JSON response)

함수가 돌려준 딕셔너리가 그대로 JSON 응답 본문이 됨. 직렬화 코드를 따로 쓰지 않음.

```
return {"answer": answer}
```

POST 요청 (POST request)

본문에 데이터를 실어 보내는 HTTP 요청. 질문 문장처럼 값을 보내야 하는 호출에 씬.

```
curl -X POST localhost:8000/ask
```

개발 서버 실행 (dev server)

파일을 지정해 개발용 서버를 띄우는 명령. 이 명령을 쓰려면 **fastapi[standard]**로 설치해야 함.

```
fastapi dev main.py
```

요청 본문 규격 (request body)

pydantic 모델로 선언하면 들어온 JSON이 그 모델로 검증되어 함수 인자로 도착함.

```
class Ask(BaseModel):  
    question: str
```

한글 본문 전송 (UTF-8 body)

한글이 든 JSON은 파일에 UTF-8로 저장해 파일째 보냄. 명령줄에 한글을 직접 적으면 깨져서 전송됨.

```
--data-binary "@body.json"
```


인사말 하나에 쓰인 검색

무조건 검색하는 구조

[질문] 안녕하세요!

→ 검색 실행

score=1.4526 외국어 안내 조각

score=1.5542 생일 혜택 조각

인사말에도 문서를 뒤져 무관한 조각 두 개가 프롬프트에 공급됨. 검색 비용과 소요 시간이 질문의 성격과 무관하게 나감.

판단해 검색하는 구조

[질문] 안녕하세요!

→ 검색 0회, 즉답

같은 질문에 검색이 일어나지 않음. 검색 도구는 등록되어 있으나 모델이 부르지 않기로 판단함.

문제상황과 목표

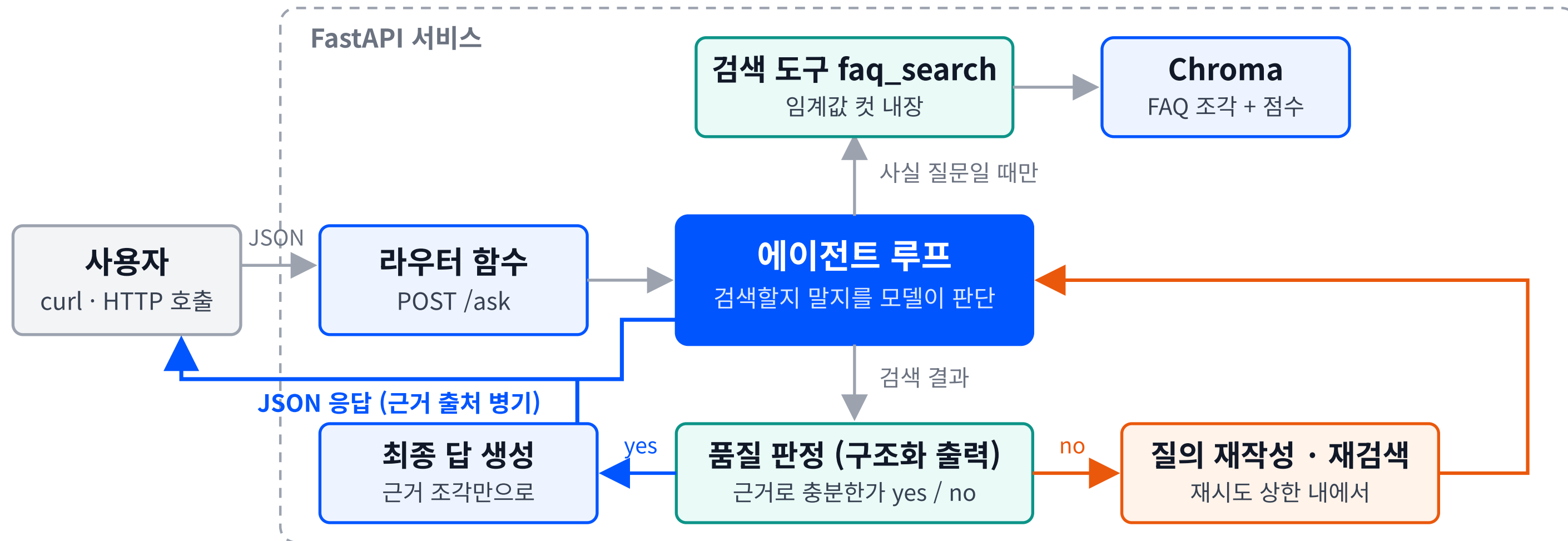
문제

안내 프로그램이 FAQ에 없는 것도 아는 척 답하고 있는데 $\pi\pi$ 홈페이지·앱 팀에서는 이 프로그램을 사용하고 싶어해요.

목표

사실을 묻는 질문일 때만 FAQ를 찾아보고 근거가 없으면 모른다고 말하는 안내 담당자를, 홈페이지·앱에서 주소 하나로 불러 쓰는 서비스로 만들어봅시다.

RAG 에이전트 서비스의 전체 골격



각 단계별 요구사항

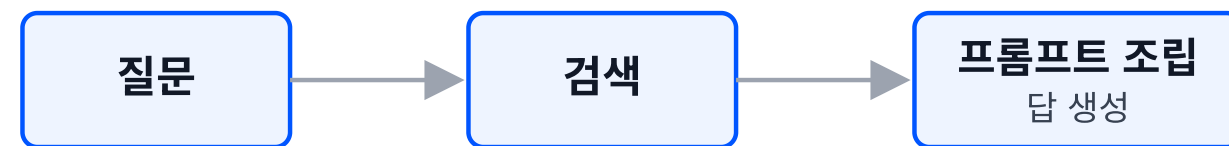
구성 요소	무엇을 받아	어떻게	무엇을 넘긴다
사용자	(흐름의 시작)	질문을 HTTP 요청으로 보내고	JSON 응답을 받는다
라우터 함수	요청의 question을	그대로 에이전트 루프에 넘기고	답을 JSON으로 돌려준다
에이전트 루프	고정 지침(시스템 프롬프트)과 질문을	도구 사용(faq_search)이 필요한 사실 질문인지 스스로 판단해	근거 자료로 최종 답을 넘긴다
검색 도구 faq_search	검색 질의문을	임계값 안의 FAQ 조각만 골라	조각을 넘기고, 없으면 「검색 결과 없음」을 넘긴다
Chroma	질의 벡터를	가까운 조각을 찾아	조각과 점수를 넘긴다
품질 판정 (구조화 출력)	질문과 검색 결과를	근거로 충분한지 yes/no로 판단해	yes면 근거만으로 답을 만들고, no면 재작성으로 보낸다
질의 재작성 · 재검색	부족 판정을	질의를 고쳐	재시도 상한 안에서 다시 검색한다

검색 함수에 이름과 설명, 파라미터 규격을 붙여 에이전트의 도구 목록에 넣는 것: 검색기의 도구화(retriever tool)

도구 목록에 들어간 순간부터 검색할지 말지의 판단은 모델의 몫이 됨. 사람이 짠 코드에는 그 분기가 없음.

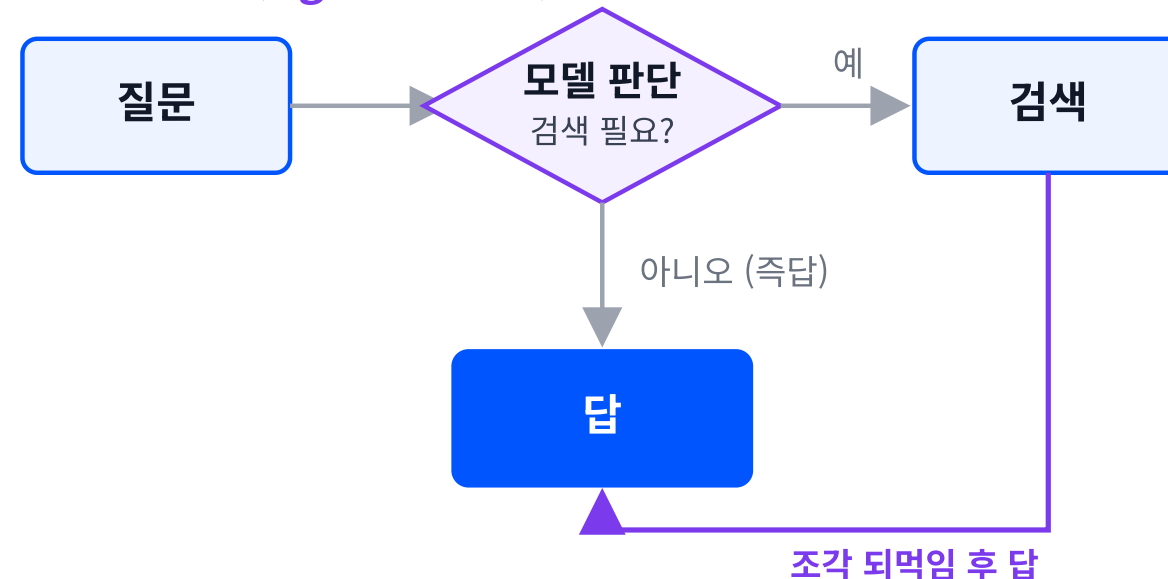
검색 시점을 정하는 두 구조

무조건 검색



분기 없음. 인사말에도 검색이 실행됨

판단해 검색 (Agentic RAG)



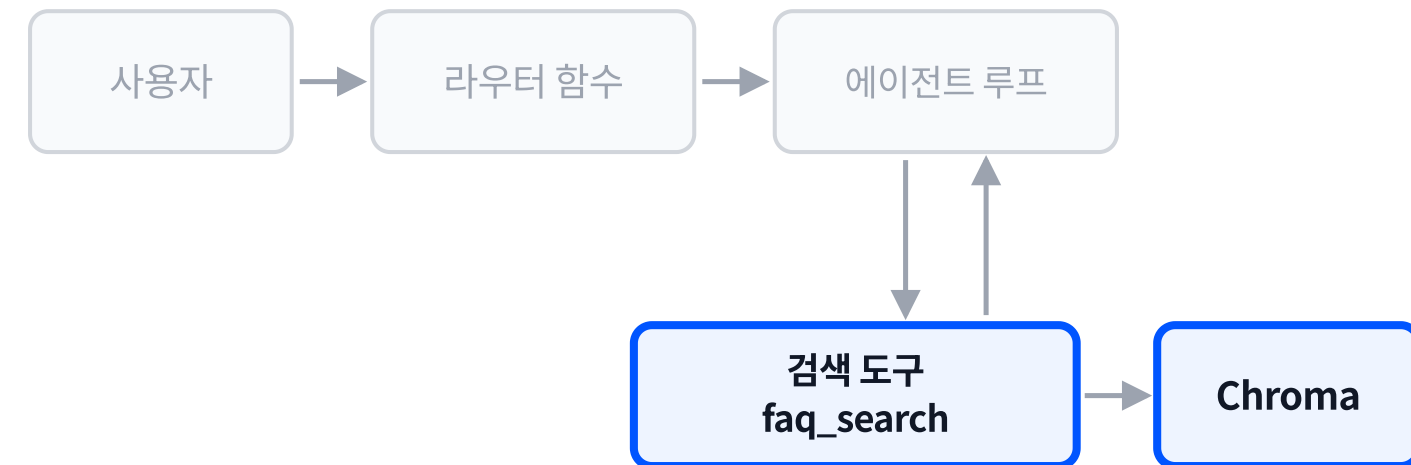
조각 되먹임 후 답

같은 「검색」 상자가 위에서는 통과 지점, 아래에서는 선택 지점.

- **무조건 검색**: 질문의 성격과 무관하게 검색을 먼저 실행하고 그 결과를 프롬프트에 넣는 구조. 분기가 코드에 없음.
- **판단해 검색 (Agentic RAG)**: 검색 시점과 질의를 모델이 정하는 구조. 세 요소로 이루어짐.
- 검색 여부 판단 / 검색 결과의 품질 판정(grade) / 불합격 시 질의 재작성 후 재검색.
- 판정과 재검색이 붙으면 검색은 한 번으로 끝나지 않고, 답이 설 때까지 되풀이됨.

1단계: 임계값 컷을 품은 검색 함수

```
def make_search_tool(db):
    @tool(parse_docstring=True)
    def faq_search(query: str) -> str:
        """시설 FAQ 문서에서 [...]"""
        hits = db.similarity_search_with_score(
            query, k=2)
        good = [(d, s) for d, s in hits
                if s <= THRESHOLD]
        if not good:
            return ("검색 결과 없음 [...] 질의를 "
                    "바꿔 다시 검색하거나, "
                    "모른다고 답하라.")
        return "\n---\n".join(
            f"[score {s:.2f}] {d.page_content}"
            for d, s in good)
    return faq_search
```



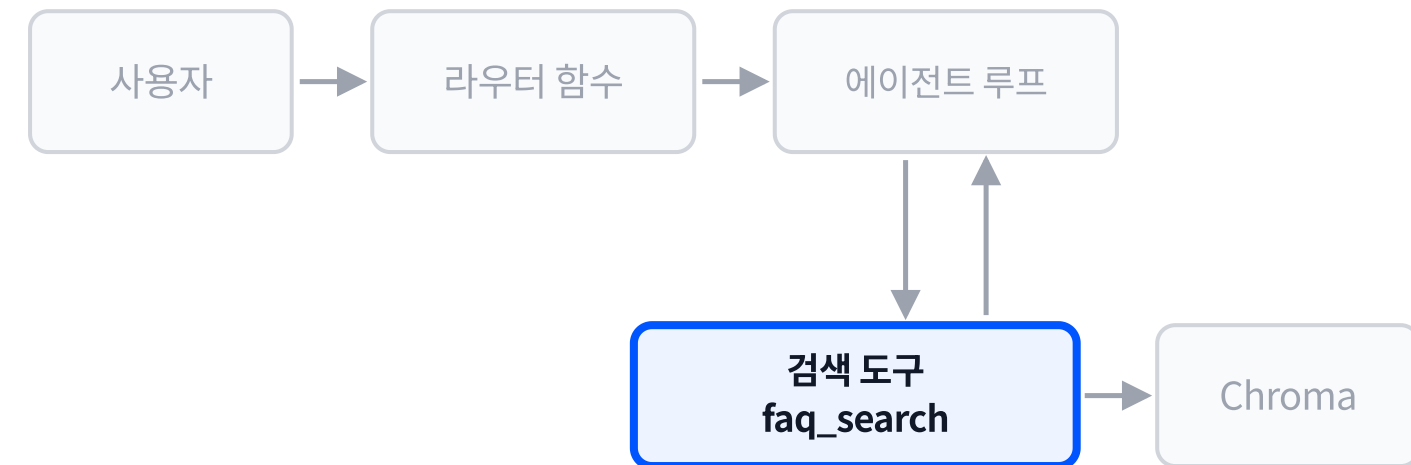
faq_search의 본체는 질의 문자열 하나를 받아 문자열 하나를 돌려주는 평범한 파이썬 함수. **@tool**이 이 함수를 도구로 감싸는 일은 다음 단계에서 봄.

THRESHOLD 이하의 조각만 남김. 거리 점수는 작을수록 가까움.

임계값 컷이 도구 안에 들어 있어, 컷 여부를 모델이 판단하지 않고 코드가 정함.

2단계: @tool로 도구 선언

```
def make_search_tool(db):  
    @tool(parse_docstring=True)  
    def faq_search(query: str) -> str:  
        """시설 FAQ 문서에서 질문과 관련된  
        조각을 검색한다. 시설 이용, 요금,  
        환불, 운영 관련 질문에 쓴다.  
  
        Args:  
            query: 검색할 질의문. 결과가 없으면  
                표현을 바꿔 재검색할 수 있다.  
        """  
        [... 임계값 컷 검색 본체 ...]  
    return faq_search
```



@tool이 함수 이름과 타입 힌트와 독스트링에서 도구 규격을 만들어 줌. 손으로 쓰던 JSON 스키마 딕셔너리가 사라짐.

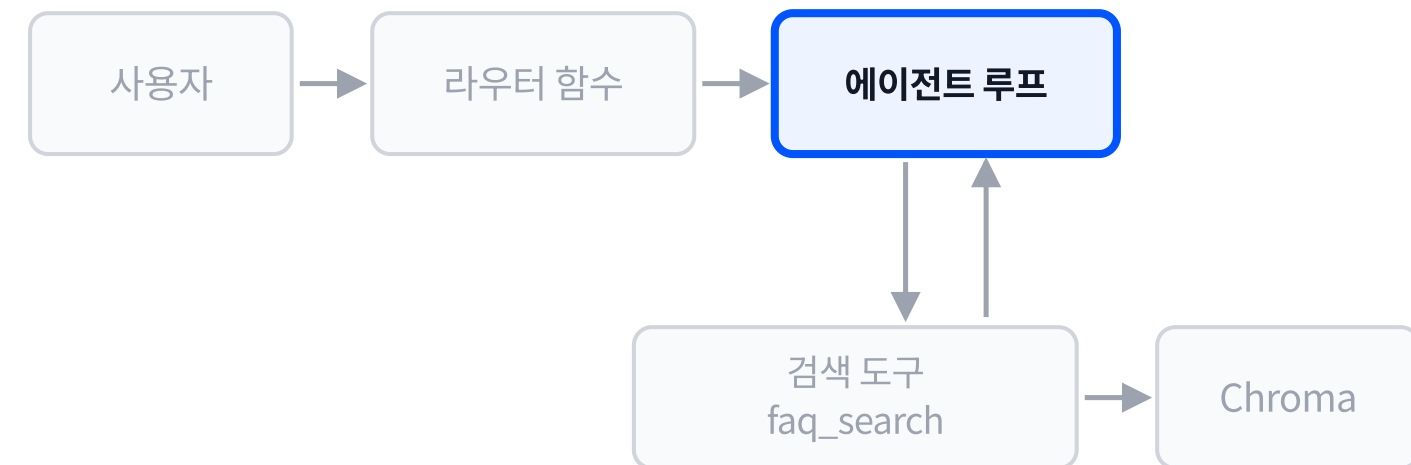
독스트링 첫 단락이 도구 설명(description)이 되고, 그 문장이 검색 판단의 실질 기준. **parse_docstring=True**면 Args의 query 설명도 파라미터 설명으로 올라감.

검색이 빈 경우 도구가 돌려주는 문장이 다음 수를 지시함.

3단계: bind_tools로 도구를 쥔 모델 호출

```
def run_rag_agent(question, db, max_turn=5):
    faq_search = make_search_tool(db)
    llm = init_chat_model("gpt-5.6-luna",
                          model_provider="litellm")
    llm_tools = llm.bind_tools([faq_search])
    system = "너는 시설 안내 담당자다. [...]"
    messages = [SystemMessage(content=system),
                HumanMessage(content=question)]

    for _ in range(max_turn):
        res = llm_tools.invoke(messages)
        if not res.tool_calls:
            return res.content
        messages.append(res)
        for call in res.tool_calls:
            [... 도구 실행·되먹임 ...]
    return "반복 한도 초과"
```

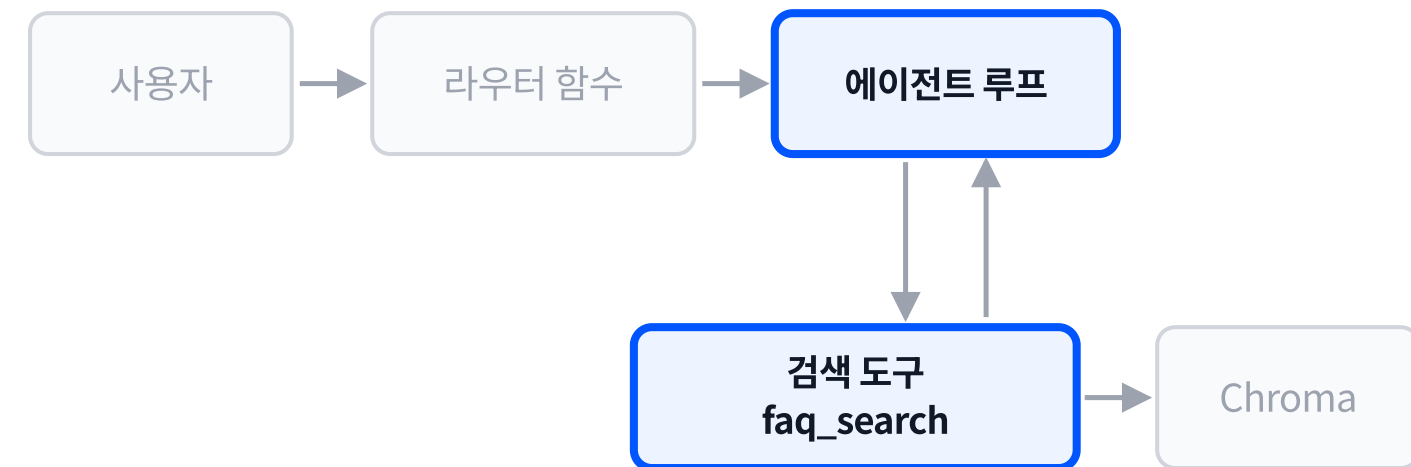


system 전문: "너는 시설 안내 담당자다. 사실 질문에는 faq_search 도구로 근거를 찾아 답한다. 인사말 등 검색이 필요 없는 말에는 바로 답한다. 근거가 없으면 모른다고 답한다."

bind_tools가 도구 목록을 모델에 묶고, 그 뒤로는 **llm_tools.invoke** 한 줄이 도구 목록과 함께 모델을 부름. 도구 목록만 갈아 끼우면 같은 루프가 다른 에이전트가 됨.

4단계: 검색 실행 결과 되먹임

```
res = llm_tools.invoke(messages)
if not res.tool_calls:
    return res.content
messages.append(res)
for call in res.tool_calls:
    result = faq_search.invoke(
        call["args"])
    messages.append(ToolMessage(
        content=result,
        tool_call_id=call["id"]))
```



tool_calls가 비어 있으면 그 자리에서 답을 반환함. 검색 없이 끝나는 질문이 이 경로로 나감.

모델의 요청과 도구 결과를 짝지어 기록에 붙임. 도구 결과는 **ToolMessage**로 붙고, 다음 호출은 그 기록을 통째로 다시 봄.

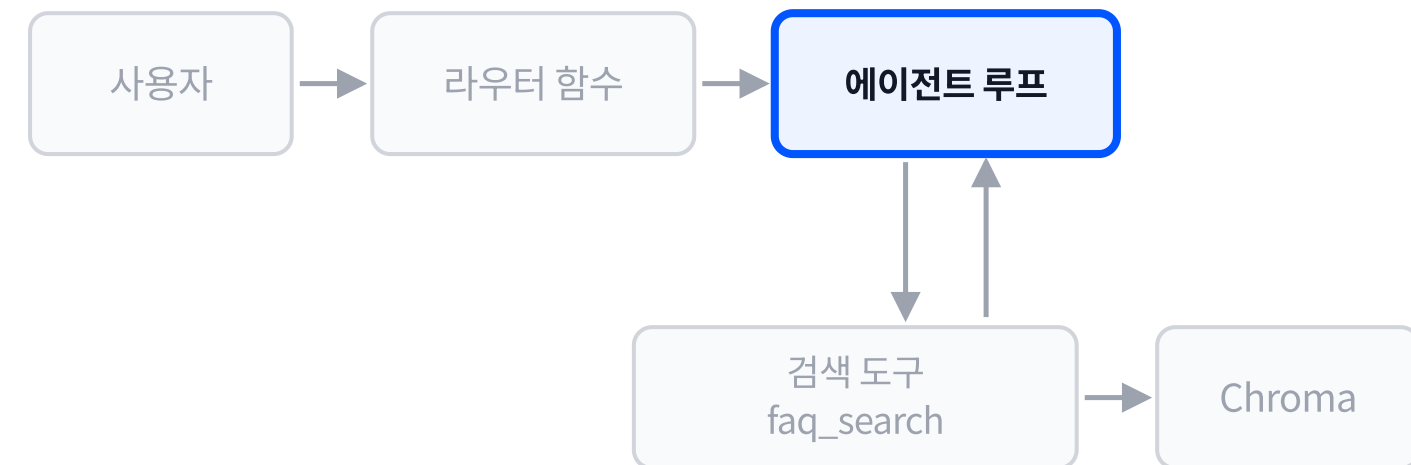
검색 결과 문자열이 기록에 실려 들어가는 것이 근거 공급의 실체.

5단계: 재시도 상한과 종료

```
def run_rag_agent(question, db, max_turn=5):
    faq_search = make_search_tool(db)
    llm = init_chat_model("gpt-5.6-luna",
                          model_provider="litellm")
    llm_tools = llm.bind_tools([faq_search])
    system = "너는 시설 안내 담당자다. [...]"
    messages = [SystemMessage(content=system),
                HumanMessage(content=question)]

    for _ in range(max_turn):
        res = llm_tools.invoke(messages)
        if not res.tool_calls:
            return res.content
        messages.append(res)
        for call in res.tool_calls:
            [... 도구 실행·되먹임 ...]

    return "반복 한도 초과"
```



max_turn이 재검색의 상한. 검색이 계속 비면 모델은 질의를 바꿔 다시 부르는데, 상한이 없으면 그 재질의가 끝나지 않음.

상한에 닿으면 루프는 답 대신 정해진 문자열을 돌려주고 멈춤. 이 상한이 서비스가 응답 없이 매달리는 상태를 막음.

안내 담당자의 고정 지침 전문

고정 지침 - 안내 담당자

너는 시설 안내 담당자다.

사실 질문에는 `faq_search` 도구로
근거를 찾아 답한다.

인사말 등 검색이 필요 없는 말에는
바로 답한다.

근거가 없으면 모른다고 답한다.

모델에 들어가는 네 총

- **고정 지침**: 왼쪽의 네 문장. 검색할지 말지의 기준과 근거 없을 때의 태도
- **도구 사용**: 도구 설명과 인자 설명(다음 지면)
- **근거 자료**: 도구가 돌려준 조각(없으면 「모른다」)
- **대화 이력**: 질문과 도구 호출·결과가 쌓이는 자리

고정 지침은 매 호출 그대로 들어가고, 검색할지 말지와 근거 없을 때의 태도를 이 네 문장이 정함.

모델이 읽는 도구 설명과, 검색에 실패했을 때 도구가 돌려주는 문장

도구 사용 - 도구 설명

시설 FAQ 문서에서 질문과 관련된 조각을 검색한다. 시설 이용, 요금, 환불, 운영 관련 질문에 쓴다.

도구 사용 - 인자 query 설명

검색할 질의문. 결과가 없으면 표현을 바꿔 재검색할 수 있다.

근거 자료 - 검색 실패 시 반환문

검색 결과 없음 (최고 유사도 점수 {점수}가 임계값 {THRESHOLD}를 넘음).
질의를 바꿔 다시 검색하거나,
모른다고 답하라.

{점수}·{THRESHOLD} 자리에는 실행 시 숫자가 들어감(임계값은 1.5).

「모른다고 답한다」가 고정 지침과 도구 반환문 두 곳에 있어, 검색이 비었을 때 모델이 답을 지어내지 못하게 이중으로 막음.

세 질문의 실행 결과

[질문] 안녕하세요!

→ 검색 0회, 즉답

[질문] 자유이용권 환불 규정

→ 검색 1회(score 0.71 정답 조각)

→ 전날 전액/당일 50%/개시 후 불가

[질문] 야간개장 때 퍼레이드 하나요?

→ 검색 1회(score 0.80)

→ 20:30 야간 퍼레이드 답

환불 질문의 전문 트레이스

[검색 호출] {'query': '자유이용권 환불
규정 환불 기준 취소 수수료'}

[검색 결과] [score 0.71] [티켓]

Q: 자유이용권 환불 규정이 어떻게
되나요? A: 이용일 전날까지 취소하면
전액 환불되고 ...

트레이스의 줄	코드의 자리
검색 0회	if not res.tool_calls 첫 바퀴 반환
검색 호출	res.tool_calls 의 항목
score 0.71	임계값 이하라 조각이 통과
최종 답	되먹임 뒤 두 번째 호출의 반환

같은 도구 목록·같은 루프에서 검색 횟수가 질문마다
다름

검색 실패의 처분: 재검색과 모름

```
class Grade(BaseModel):  
    sufficient: Literal["yes", "no"]  
  
grader = llm.with_structured_output(Grade)  
verdict = grader.invoke(  
    f"질문: {question}\n검색 결과:\n"  
    f"{context}\n이 검색 결과가 질문에 "  
    "답할 근거로 충분한가?").sufficient
```

판정	다음 수
조각이 질문에 답함	그 조각의 내용으로 답을 만듦
조각이 질문에 못 미침	질의를 고쳐 재검색
상한에 닿음	모른다고 답하고 종료

- 판정을 산문으로 받으면 분기 코드가 문장을 해석해야 함. **두 값 중 하나만 담는 규격**으로 받으면 분기가 값 하나를 보고 나뉨.
- 재작성·재검색에는 **상한이 반드시 붙음**. 상한 없는 재질 의는 종료 조건이 없음.

교정 재검색(Corrective RAG, CRAG)의 실행 결과

[질문] 겨울에 눈썰매장 운영하나요?

[판정 1회차] query='겨울에 눈썰매장 운영하나요?' → no

[판정 2회차] query='눈썰매장은 겨울철에만 운영되나요? 운영 기간은 언제인가요?' → no

[판정 3회차] query='눈썰매장의 운영 여부와 이용 가능한 운영일 및 운영시간을 알려주세요.' → no

[최종 답] 문서에서 근거를 찾지 못했습니다. 안내 창구로 문의해 주세요.

- **no 판정**: 가져온 조각이 답의 근거로 부족하다는 뜻.
- 판정이 no로 나오자 **질의가 다른 표현으로 재작성**되어 재검색이 일어남. 판정 2회차의 query는 에이전트가 질문을 교정(Correction)하여 만든 것. 회차마다 앞 질의와 다른 각도로 다시 교정함.
- 상한(2회)에 닿자 지어낸 답 대신 **정해진 폴백 문장**으로 종료함. 근거 없는 답이 나가는 길이 코드로 막혀 있음.

에이전트를 HTTP 주소로 노출하기

```
app = FastAPI()

class AskIn(BaseModel):
    question: str

class AskOut(BaseModel):
    answer: str

@app.post("/ask")
def ask(body: AskIn) -> AskOut:
    return AskOut(answer=run_rag_agent(
        body.question, db))
```

- 요청 본문의 칸과 타입은 **pydantic 모델**이 못 박고, 라우터 함수는 주소 하나에 함수 하나를 매어 둠.
- 함수 안에서 손 루프를 그대로 부르고, 반환한 값이 **JSON 응답 본문**이 됨.

```
$ curl -X POST 127.0.0.1:8000/ask
-H "Content-Type: application/json"
--data-binary "@body_ask.json"
{"answer": "자유이용권 환불 규정은 다음과 같습니다. [...]"}

```

에이전트 코드는 손대지 않음. 감싸는 함수 하나가 붙을 뿐.

조각에 실어 둔 출처

```
for i, row in enumerate(csv.DictReader(f),
                                start=1):
    text = f"[{row['Category']}] "\
           f"Q: {row['Question']}\n"\
           f"A: {row['Answer']}"
    docs.append(Document(page_content=text,
                        metadata={"row": i,
                                "category": row["Category"]})))
```

[질문] 자유이용권 환불 규정 알려 주세요
[답] 이용일 전날까지 취소하면 전액
환불되며, 이용일 당일 취소 시 50%만
환불됩니다. 이용 개시 후에는 환불되지
않습니다. (근거: 8행)

답변에 출처를 적으려면 조각에 출처가 실려 있어야 함

metadata 칸	값의 예	답변에서의 쓰임
row	원본 표의 행 번호	어느 줄에서 왔는지 지목
category	환불·운영·시설	어느 항목의 규정인지 표시

- 저장 시점에 넣지 않은 정보는 검색 결과에서 되찾을 수 없음. 출처 병기의 준비는 적재 코드에서 끝남.
- 검색 결과 객체는 조각 본문과 이 metadata를 함께 돌려줌.

"좋은 검색"을 데이터로 적기

평가셋(evaluation set) 한 건의 칸	값
질문	자유이용권을 이용 당일에 환불할 수 있는지
기대 근거	구름월드 FAQ의 환불 항목 조각
판정	검색 상위 세 조각 안에 기대 근거가 있는지 (있음 / 없음)

질문과 기대 근거의 쌍을 여러 건 적어 두면, 임계값이나 청크 크기를 바꿀 때마다 같은 잣대로 잴 수 있음

검색 품질은 답변 문장의 인상이 아니라 기대 근거의 적중 여부로 잴. 적중 비율(hit rate@k · recall@k)과 임계값이 잘못 걸러 낸 건수가 최소 지표.

검색 품질을 올리는 확장 기법

기법	정의	언제 꺼내나
질의 재작성 (Query Rewriting)	검색이 빈 질의를 다른 표현으로 고쳐 다시 검색하는 방법	사용자의 말과 문서의 어휘가 어긋나 1차 검색이 비는 경우
재정렬 (Reranking)	1차 검색으로 넉넉히 가져온 조각을 별도 모델로 다시 채점해 순서를 바꾸는 방법	상위 조각 안에 정답이 있으나 1위로 올라오지 않는 경우
하이브리드 검색 (Hybrid Search)	키워드 일치 점수와 벡터 유사도 점수를 합쳐 순위를 매기는 방법	제품 코드나 고유명사처럼 철자 일치가 중요한 질의가 섞이는 경우
질의 응축 (Query Condensation)	앞말에 기대는 후속 질문을 대화 기록으로 독립 질문으로 고쳐 쓴 뒤 검색하는 방법	대화를 이어가는 서비스에 "그건 얼마예요?"류 질문이 들어오는 경우

네 기법은 모두 검색기와 에이전트 사이에 끼워 넣는 부품. 필요한 지점을 골라 하나씩 넣고, 넣기 전후를 평가셋으로 비교함.

실습합시다

챕터 요약

- ✓ 검색기의 도구화: 검색 함수에 이름·설명·파라미터를 붙여 도구로 넣으면 검색 여부를 모델이 도구 설명을 기준으로 판단함
- ✓ Agentic RAG(grade·교정 재검색, CRAG): 검색 결과의 적합 여부를 합격·불합격 두 값으로만 받는 규격으로 판정하고, 불합격이면 질의를 고쳐 재검색하며, 재검색에는 상한이 붙음
- ✓ 출처 병기(metadata): 답변에 근거의 출처를 병기하려면 적재 시점에 조각의 metadata에 출처를 실어 두어야 함
- ✓ 확장 기법(질의 재작성, 재정렬, 하이브리드, 질의 응축)은 검색 품질을 올리는 기법이고, 평가셋으로 넣기 전후를 비교해 고름
- ✓ FastAPI 노출: 손 루프를 라우터 함수로 감싸면 HTTP 주소로 호출하는 서비스가 됨

Agent 개요

RAG와 LLM wiki

전역 질문 / 검색과 연결 / 위키 사용법

학습 목표

- RAG과 llm-wiki의 동작 차이를 설명할 수 있습니다.
- Karpathy 원문으로 자기 위키 운영 규칙을 작성하고, '반입, 질의, 기록'을 거쳐 llm-wiki를 사용할 수 있습니다.
- 어떤 상황에 RAG을 쓰고 어떤 상황에 llm-wiki를 쓰는지 예를 들어 설명할 수 있습니다.

- ✓ 에이전트 동작 원리
- ✓ 랭체인 기초
- ✓ RAG: 검색기 구축
- ✓ RAG 에이전트 서비스
- ▶ **RAG와 LLM wiki**

같은 검색기에 던진 두 질문

질문 A 자유이용권 환불이 되나요?

score=0.6878 | [티켓] Q: 자유이용권 환불 규정이 어떻게 되나요? ...

score=0.9641 | [티켓] Q: 자유이용권 종류가 어떻게 되나요? ...

score=1.0076 | [티켓] Q: 연간이용권 혜택이 궁금합니다. ...

답이 1위 조각 안에 있음

질문 B 이 문서 전체를 관통하는 주제 3개는 무엇인가요?

score=1.5702 | [편의시설] Q: 물품보관함이 있나요? ...

score=1.6107 | [서비스] Q: 외국어 안내 서비스가 있나요? ...

score=1.6387 | [분실물] Q: 물건을 잃어버렸어요. 어디에 문의하나요? ...

무관한 개별 조각 3개만 반환됨. 나머지 29행은 답에 참여하지 못함

조각 검색이 답하지 못하는 질문 두 종

multi-hop QA

"징검다리가 필요한 질문"

답이 두 개 이상의 문서에 나뉘어 있어, 조각을 이어야만 답이 되는 질문. 조각 하나하나는 답의 절반씩만 담고 있어 유사도 상위에 오르지도 않음.

global sensemaking

"전체를 봐야 하는 질문"

문서 더미 전체를 봐야 답이 되는 질문. 주제 정리, 흐름 요약, 반복되는 불만 유형 집계가 여기 속함. top-k는 k개만 반환하므로 원리상 전체에 닿지 못함.

원문의 두 문장

매 질문마다 처음부터

```
the LLM is rediscovering  
knowledge from scratch on  
every question. There's no  
accumulation.
```

질문이 들어올 때마다 조각을 다시 찾아 다시 잇는 구조. 같은 질문을 두 번 해도 두 번 모두 처음부터 시작됨.

남아서 불어나는 문서

```
the wiki is a persistent,  
compounding artifact.
```

자료가 들어올 때 이어 둔 결과가 파일로 남는 구조.
자료가 늘고 질문이 쌓일수록 이미 정리된 몫이 커짐.

검색과 연결

검색 (retrieval)

"가까운 조각을 고른다"

질문이 들어온 뒤 의미 좌표가 가까운 조각을 고르는 연산. 고른 조각을 프롬프트에 넣을 뿐, 조각과 조각 사이를 잇지 않음. 잇는 일은 매번 모델이 즉석에서 함.

연결 (linking)

"미리 이어 둔다"

자료가 들어올 때 기존 문서에 이어 붙이는 편집. 상호 참조와 모순 표시가 편집 시점에 문서로 박힘. 질문이 오기 전에 이미 이어져 있음.

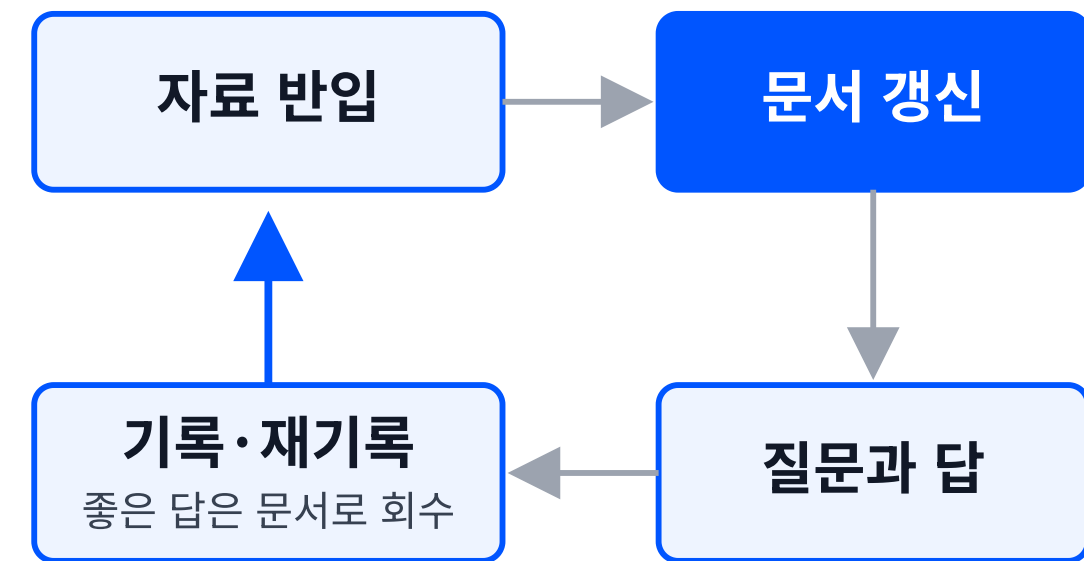
RAG과 llm-wiki의 순환 모양

검색: 열린 사슬



답은 채팅에서 휘발됨. 다음 질문은 다시 처음부터.

위키: 닫힌 고리



한 바퀴 돌 때마다 문서가 남음

왼쪽은 끝나는 사슬, 오른쪽은 도는 고리. 남는 것의 유무가 두 방식의 차이.

RAG vs llm-wiki

	강점	실패하는 경우	적합한 상황
RAG	질의 시점에 가까운 조각을 고르므로, 정리해 두지 않은 문서 더미에도 바로 물을 수 있음. 문서가 바뀌면 색인만 다시 만들면 됨.	답이 여러 문서에 흩어져 이어 붙여야 하는 질문(multi-hop QA), 문서 더미 전체를 봐야 하는 질문(global sensemaking)에 top-k가 닿지 못함. 같은 질문을 다시 물어도 매번 처음부터 찾음.	답이 한두 구절에 들어 있는 사실 조회. 자주 바뀌어 정리해 둘 새가 없는 정보. 아무도 읽지 않은 대량 문서 더미.
llm-wiki	투입 시점에 이어 두므로 상호 참조와 모순 표시가 문서로 남음. 질문이 쌓일수록 이미 정리된 몫이 커짐.	맞고 틀림을 판정하는 사람이 없으면 틀린 정리가 그대로 굳음. 자동 요약이 문서 사이의 모순을 뭉갤 위험. 만들어 두고 열지 않으면 자산이 되지 못함.	반복해서 묻게 되는 주제. 전체를 봐야 하는 질문. 정리 결과를 확인해 줄 사람이 있는 자리.

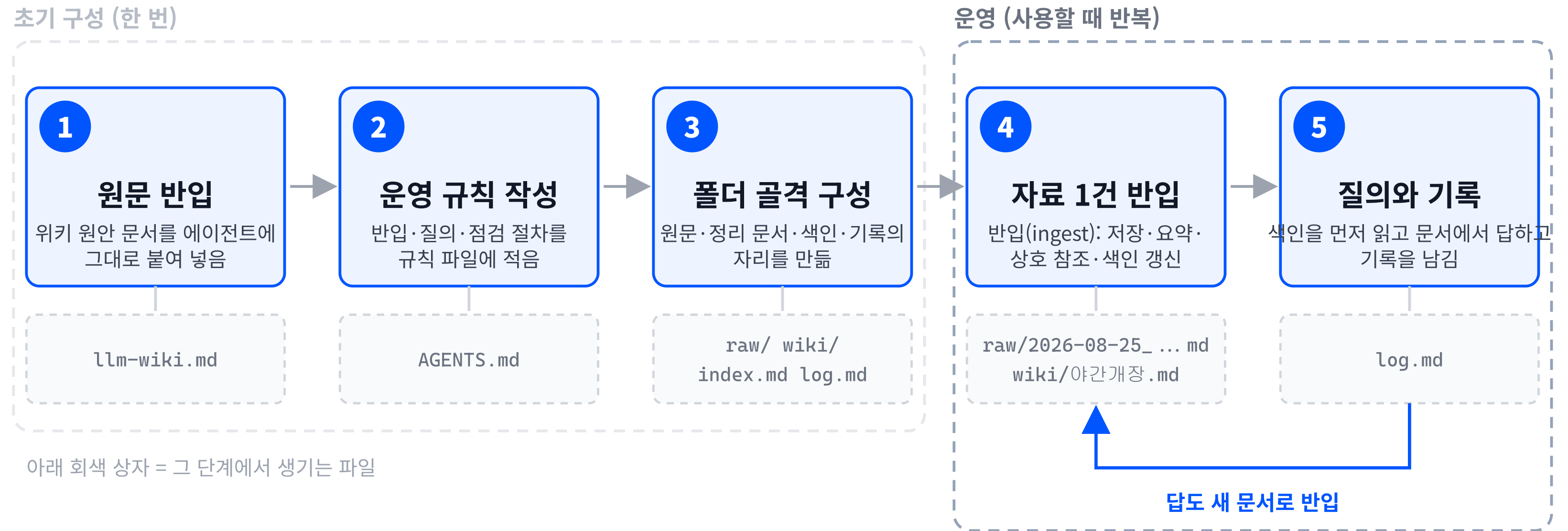
질문 유형별 선택 기준

질문 유형	묻는 곳
한두 구절에 답이 있는 사실 조회	검색
자주 바뀌는 최신 정보	검색
읽지 않은 대량 문서 더미	검색
두 문서 이상을 이어야 하는 질문	위키
전체를 봐야 하는 질문	위키
반복해서 다시 묻게 되는 질문	위키

두 방식은 서로를 대체하지 않음. 같은 자료를 두고 검색기와 위키를 함께 둘 수 있음

- 검색기는 **조회 창구**로 두고, 위키는 **추적 장부**로 둬
- 판단 기준은 질문의 성질. 답이 한자리에 있으면 검색, 여러 자리를 이어야 하면 위키

위키 운영 워크플로우



원문 전문 (1/3): 핵심 발상

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM incrementally builds and maintains a persistent wiki – a structured, interlinked collection of markdown files that sits between you and the raw sources.

When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki – updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then kept current, not re-derived on every query.

This is the key difference: the wiki is a persistent, compounding artifact. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

원문 해설 (1/3): 축적의 유무

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM **is rediscovering knowledge from scratch on every question. There's no accumulation.**

매번 처음부터 벡터 검색은 질문을 받을 때마다 조각을 다시 고름. 답이 끝나면 남는 것이 없어, 같은 질문을 다시 받아도 처음부터 다시 함.

Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** – a structured, interlinked collection of markdown files that sits between you and the raw sources.

This is the key difference: the wiki is a persistent, compounding artifact.

쌓이는 자산 자료가 들어올 때마다 에이전트가 읽고 기존 문서에 반영함. 지식이 한 번 정리된 뒤로는 갱신만 되며, 문서가 그대로 남음.

원문 전문 (2/3): 세 층

There are three layers:

Raw sources – your curated collection of source documents. Articles, papers, images, data files. These are immutable – the LLM reads from them but never modifies them. This is your source of truth.

The wiki – a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons, an overview, a synthesis. The LLM owns this layer entirely. It creates pages, updates them when new sources arrive, maintains cross-references, and keeps everything consistent. You read it; the LLM writes it.

The schema – a document (e.g. CLAUDE.md for Claude Code or AGENTS.md for Codex) that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki. This is the key configuration file – it's what makes the LLM a disciplined wiki maintainer rather than a generic chatbot. You and the LLM co-evolve this over time as you figure out what works for your domain.

원문 해설 (2/3): 층마다 다른 소유자

Raw sources – your curated collection of source documents. **These are immutable – the LLM reads from them but never modifies them.**

The wiki – a directory of LLM-generated markdown files. **The LLM owns this layer entirely.**
You read it; the LLM writes it.

The schema – a document (e.g. CLAUDE.md for Claude Code or AGENTS.md for Codex) **that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki.**

원문 보관 받은 자료를 고치지 않고 그대로 두는 층. 답의 근거를 이 층의 파일명으로 지목함.

정리 문서 에이전트가 쓰고 사람이 읽는 층. 새 자료가 들어오면 이 층의 문서가 갱신됨.

운영 규칙 두 층을 어떤 절차로 다룰지 적어 둔 파일. 이 파일이 있어야 절차가 매번 같은 순서로 돌아감.

원문 전문 (3/3): 반입과 질의

Ingest. You drop a new source into the raw collection and tell the LLM to process it. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10–15 wiki pages. Personally I prefer to ingest sources one at a time and stay involved – I read the summaries, check the updates, and guide the LLM on what to emphasize. But you could also batch-ingest many sources at once with less supervision. It's up to you to develop the workflow that fits your style and document it in the schema for future sessions.

Query. You ask questions against the wiki. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. Answers can take different forms depending on the question – a markdown page, a comparison table, a slide deck (Marp), a chart (matplotlib), a canvas. The important insight: good answers can be filed back into the wiki as new pages. A comparison you asked for, an analysis, a connection you discovered – these are valuable and shouldn't disappear into chat history. This way your explorations compound in the knowledge base just like ingested sources do.

원문 해설 (3/3): 반입 · 질의 · 점검

Lint. Periodically, ask the LLM to health-check the wiki. Look for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

점검 (lint) 모순된 문장 쌍, 낡은 주장, 아무 문서도 가리키지 않는 문서를 찾아내는 절차. 위키가 커질수록 사람 눈으로는 못 잡음.

Ingest. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log.

반입 (ingest) 자료 1건이 문서 10~15개를 건드림. 사람이 손으로 하기 버거워 위키가 방치되던 이 반복 작업을 에이전트가 맡음.

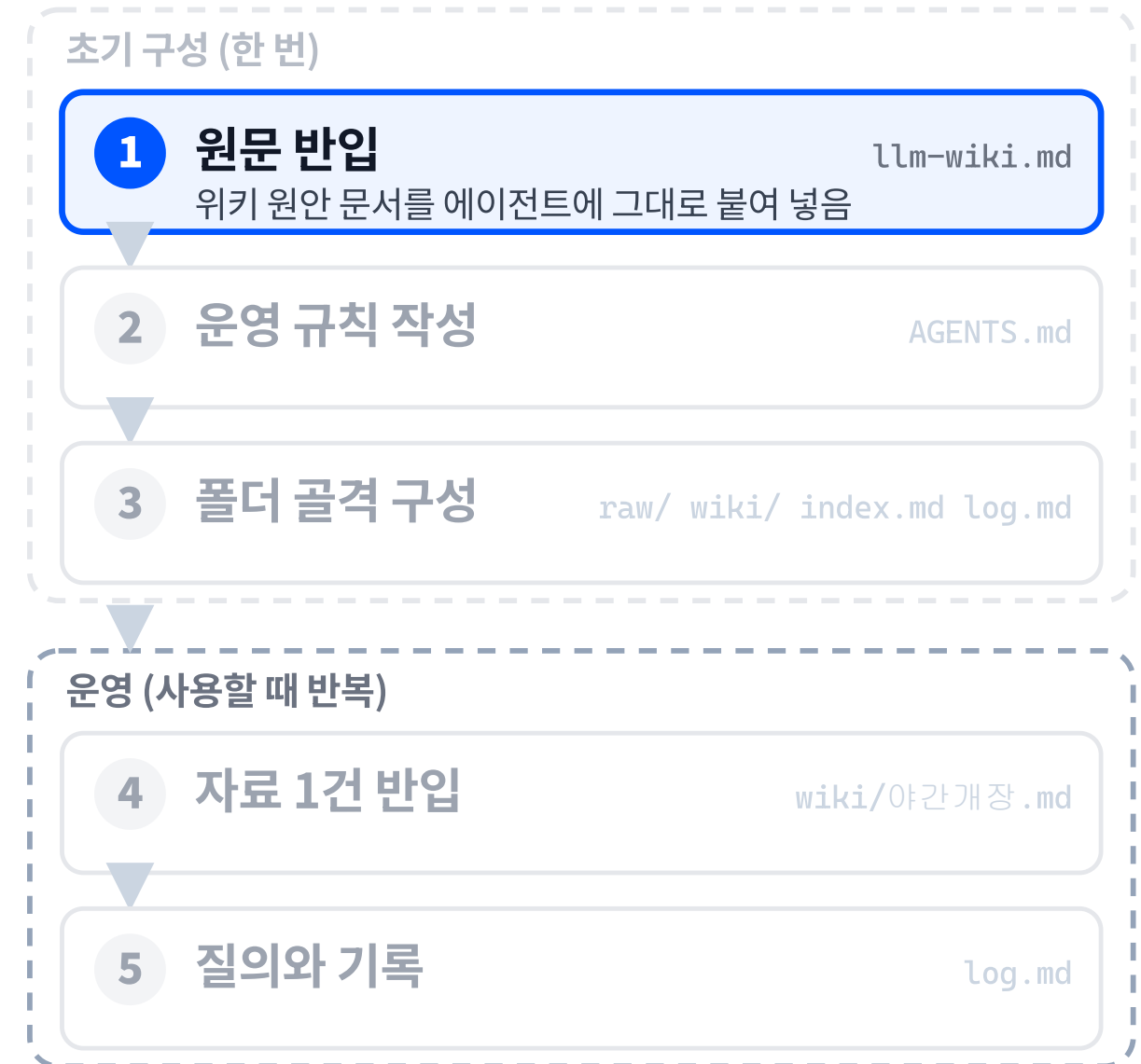
Query. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. The important insight: good answers can be filed back into the wiki as new pages.

질의 (query) 답에 근거를 함께 적음. 잘 나온 답은 채팅에 두지 않고 새 문서로 되돌려 반입함.

1단계: 원문 반입

이 문서가 위키 원문입니다. 읽어 주세요.

세 층이 필요합니다. raw, wiki, schema입니다.



이 문장들은 개념 설명이 아니라 위키 에이전트에게 그대로 주는 지시문.

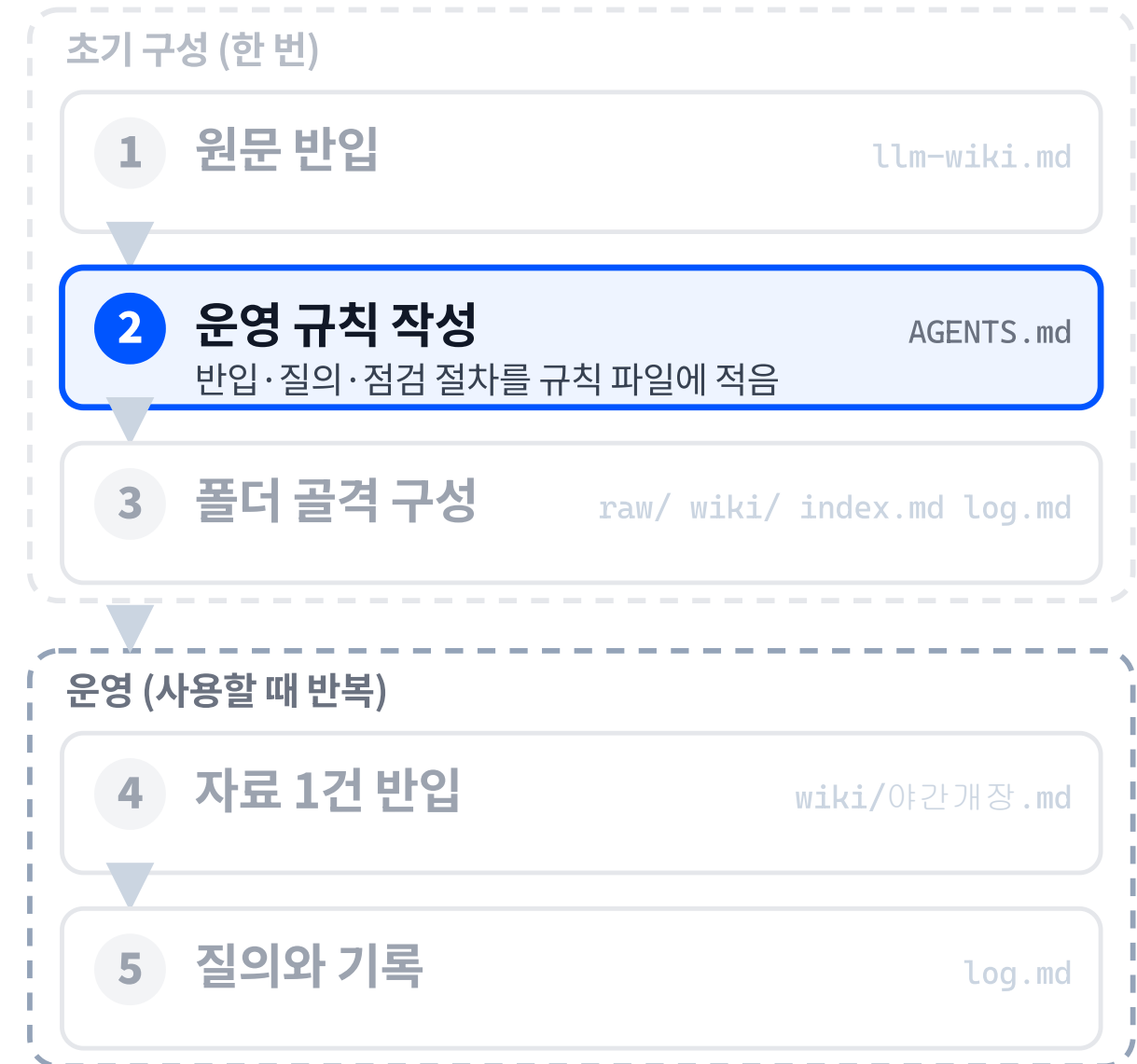
2단계: 운영 규칙 작성

이 문서가 위키 원안입니다. 읽어 주세요.

세 층이 필요합니다. raw, wiki, schema입니다.

운영 규칙 파일부터 만들어 주세요.

AGENTS.md를 썼습니다. 반입·질의·점검 3절입니다.



규칙 파일이 있어야 에이전트가 일반 챗봇이 아니라 위키 관리자로 동작함. 절차는 각자 자기 위키에 맞게 적음.

3단계: 폴더 골격 구성

이 문서가 위키 원안입니다. 읽어 주세요.

세 층이 필요합니다. raw, wiki, schema입니다.

운영 규칙 파일부터 만들어 주세요.

AGENTS.md를 썼습니다. 반입·질의·점검 3절입니다.

규칙대로 폴더를 세워 주세요.

raw/, wiki/, index.md, log.md를 만들었습니다.



폴더 골격은 에이전트가 만듦. 사람의 몫은 어떤 분류를 둘지 정하는 것.

4단계: 자료 1건 반입

이 문서가 위키 원안입니다. 읽어 주세요.

세 층이 필요합니다. raw, wiki, schema입니다.

운영 규칙 파일부터 만들어 주세요.

AGENTS.md를 썼습니다. 반입·질의·점검 3절입니다.

규칙대로 폴더를 세워 주세요.

raw/, wiki/, index.md, log.md를 만들었습니다.

운영 회의 메모입니다. 반입해 주세요.

wiki/야간개장.md 신설, 색인과 기록을 갱신했습니다.

초기 구성 (한 번)

1 원문 반입

llm-wiki.md

2 운영 규칙 작성

AGENTS.md

3 폴더 골격 구성

raw/ wiki/ index.md log.md

운영 (사용할 때 반복)

4 자료 1건 반입

wiki/야간개장.md

저장·요약·상호 참조·색인 갱신을 한 번에

5 질의와 기록

log.md

원문은 raw/에 그대로 두고 정리 문서만 새로 씀. 원문을 고치지 않는 것이 이 단계의 규칙.

5단계: 질의와 기록

이 문서가 위키 원안입니다. 읽어 주세요.

세 층이 필요합니다. raw, wiki, schema입니다.

운영 규칙 파일부터 만들어 주세요.

AGENTS.md를 썼습니다. 반입·질의·점검 3절입니다.

규칙대로 폴더를 세워 주세요.

raw/, wiki/, index.md, log.md를 만들었습니다.

운영 회의 메모입니다. 반입해 주세요.

wiki/야간개장.md 신설, 색인과 기록을 갱신했습니다.

야간개장 연장은 언제부터입니까?

9월 첫 주말부터 금·토 23:30입니다. 근거는 회의 메모입니다.

초기 구성 (한 번)

1 원문 반입

llm-wiki.md

2 운영 규칙 작성

AGENTS.md

3 폴더 골격 구성

raw/ wiki/ index.md log.md

운영 (사용할 때 반복)

4 자료 1건 반입

wiki/야간개장.md

5 질의와 기록

색인을 먼저 읽고 문서에서 답하고 기록을 남김

log.md

답을 찾지 못하면 일반 지식으로 메우지 않고 없음을 기록함.
다음에 무엇을 반입할지가 그 자리에서 정해짐.

여러 문서를 엮어 답하는 질의

야간개장 회차도 자유이용권 환불 규정이 같습니까?

야간개장 회차도 같은 환불 규정을 따릅니다. 이용일 전날까지 취소하면 전액, 당일 취소는 50%입니다. 근거는 wiki/야간개장.md 1절과 wiki/환불규정.md 2절입니다.

```
# log.md
## [2026-08-26] query | "야간개장 회차의 환불 규정"
→ 야간개장.md 1절 + 환불규정.md 2절 병기
```

답이 한 문서에 없고 두 문서에 나뉘어 있음. 색인이 두 문서를 같이 가리키므로 에이전트가 둘을 열어 하나의 답으로 묶음.



한 바퀴 실행 기록

시작 상태

AGENTS.md 반입·질의·점검 규칙

raw/ (비어 있음)

wiki/ (비어 있음)

index.md (비어 있음)

log.md (비어 있음)

회의 메모 1건을 반입하고 두 번 물은 뒤

AGENTS.md

raw/2026-08-25_운영회의메모.md

원문 그대로, 고치지 않음

wiki/야간개장.md

근거 파일명 병기, 미결 질 포함

index.md 야간개장 · 연장 운영 결정과 ...

원문 1건

log.md 항목 3개

```
$ grep "^## \[" log.md | tail -3
```

[2026-08-25] ingest | 운영 회의 메모

야간개장 문서 신설, 색인 갱신

[2026-08-25] query | "야간개장 연장은

언제부터인가" → 야간개장.md에서 답함

(9월 첫 주말부터 금·토 23:30)

[2026-08-25] query | "겨울 눈썰매장

계획은?" → 없음 - 다음 반입 후보로 기록

반입 원문은 그대로 두고 정리 문서만 새로 생김. **질의** 답에 근거
파일이 붙음. **없음** 답이 없으면 그대로 적어 다음 반입 대상이 됨.

참고: 같은 골격에서 같아 끼우는 부품

부품	예시 코드에 쓴 이름	같아 끼우는 예	바뀌지 않는 것
도구	<code>faq_lookup</code> · <code>get_now</code> · <code>faq_search</code>	사내 API 조회, DB 질의, 계산기	도구 호출 루프와 스키마 3요소
검색기	<code>Chroma.similarity_search_with_score</code>	키워드 검색, 하이브리드, 재정렬	검색 함수의 입력(질문)과 출력(조각·점수)
적재부	CSV 한 행씩 → <code>Document</code>	PDF, 웹 페이지, 사내 위키	조각과 metadata를 벡터 저장소에 넣는 절차
모델	<code>gpt-5.6-luna</code>	다른 제공사 모델, 소형 모델	<code>init_chat_model</code> · <code>completion</code> 을 호출하는 위치
판정	<code>grade</code> (근거 충분 여부의 구조화 출력)	출처 일치 검사, 사람 승인	불합격 시 재검색으로 돌아가는 분기

실습합시다

챕터 요약

- ✓ RAG은 질의 시점에 조각을 골라 답을 만들고, llm-wiki는 투입 시점에 이어 둔 문서를 남김
- ✓ 문서 더미 전체를 봐야 하는 질문에 top-k 검색은 부분 조각만 반환함
- ✓ 답이 한두 구절에 있는 조회와 자주 바뀌는 정보에는 RAG이 유리하고, 두 방식은 대체가 아니라 병용할 수 있음
- ✓ 위키는 원문 보관(raw)·정리 문서(wiki)·운영 규칙(schema) 세 층으로 이루어지고, 운영 규칙 파일이 반입·질의·점검 절차를 정함
- ✓ 위키 운영은 원문 반입, 운영 규칙 작성, 폴더 골격 구성, 자료 반입, 질의와 기록의 다섯 단계로 돌고, 질의 기록이 쌓여야 자산이 됨

